

GraphTCN: A Geometric Extension of ModernTCN for Multivariate Time-Series Forecasting

Vishal Ruparelia

Candidate Number: VRKX8

MSc Computational Statistics and Machine Learning

Supervisors: Dr Vasileios Lampos and Dr Dimitri Malandreniotis

Submission date: 9 September 2026

Disclaimer: This report is submitted as part requirement for the MSc Computational Statistics and Machine Learning at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged.

Abstract

Multivariate financial time-series forecasting is challenging because asset prices are noisy, non-stationary, and connected through relationships that may vary over time. This dissertation investigates whether dynamic graph neural networks can improve short-horizon forecasting of large cap stocks by using graph neural networks to learn time varying dependencies between series.

We propose *GraphTCN*; a novel time series architecture that uses a *ModernTCN* [25] temporal backbone to learn intra series patterns in combination with a dynamic graph learning module to learn inter series relationships. We evaluate *GraphTCN* using one-minute stock price data for 93 assets, with forecast horizons of 1, 5, 15, 30, and 60 minutes. Comparisons are made against persistence, statistical models, ModernTCN, and a zero-shot foundational financial time series model called *Kronos* [32]. *GraphTCN* produced competitive forecasts and produced lower average error metrics than all baselines apart from a simple persistence model, demonstrating the difficulty of improving upon the latest observed price in low signal-to-noise ratio financial data.

To determine whether this outcome reflected a limitation of the architecture or the difficulty of predicting in the financial domain, *GraphTCN* was also evaluated on a spatial weather benchmark dataset [33]. With extremely limited hyper parameter tuning, *GraphTCN* achieved lower mean absolute errors than the published ModernTCN baseline in 56 of 60 forecasting experiments and ranked approximately third on average among ten evaluated models. Overall, the results indicate that *GraphTCN* can exploit structured cross-series relationships when these are sufficiently stable and informative.

Declaration

I, Vishal Ruparelia, declare that this dissertation has been composed by me and that the work submitted is my own except where the contribution of others is explicitly acknowledged and referenced. This report has not been submitted for any other degree or professional qualification.

Signature: *Vishal Ruparelia* **Date:** September 9, 2026

Contents

Abstract **i**

1 Introduction **1**

1.1 Motivation 1

1.2 Research Objectives 3

1.3 Report Overview 4

2 Background and Related Work **5**

2.1 Time-Series Forecasting 5

2.2 Geometric Deep Learning 9

2.2.1 Graphs and Message Passing 9

2.2.2 Static, Learned, and Dynamic Graphs 11

2.3 Geometric Deep Learning for Time Series 12

2.4 ModernTCN 15

2.4.1 Patch Embedding 15

2.4.2 Convolutions 16

2.5 Kronos 17

2.5.1 Tokenizer 18

2.5.2 Autoregressive Token Forecasting 19

2.6 Summary 20

3 Methodology **21**

3.1 Dataset 21

3.1.1 Train-Validation-Test Split 22

3.1.2 Train Split Analysis . . . 24
3.2 GraphTCN . . . 28
3.2.1 Temporal and State Representations . . . 29
3.2.2 Graph Learning . . . 30
3.2.3 Graph Message Passing . . . 33
3.2.4 Forecasting Head . . . 34
3.3 Experiments . . . 34
3.3.1 Forecasting Setup . . . 34
3.3.2 Baselines . . . 35
3.3.3 Evaluation Metrics . . . 38
3.3.4 Training and Optimisation . . . 41
3.3.5 Hyperparameter Selection . . . 42

4 Results and Discussion **43**
4.1 Finance Dataset . . . 43
4.1.1 Forecasting Results . . . 43
4.1.2 Graph Behaviour . . . 48
4.1.3 Ablation Study . . . 53
4.2 Weather Dataset . . . 54
4.2.1 Forecasting Results . . . 55
4.2.2 Graph Behaviour . . . 59
4.3 Summary . . . 62

5 Conclusion **64**
5.1 Main Findings . . . 64
5.2 Limitations . . . 65
5.3 Future Work . . . 66

A Code **72**

B AI Usage Statement **73**

C Weather Dataset Full Results **74**

List of Figures

2.1 Generic structure of a spatio-temporal graph forecasting model. A temporal module extracts patterns within each series, while a graph-based spatial module exchanges information between series according to a supplied or learned adjacency matrix. 13

3.1 Boxplots of average realised volatility per day over all stocks for each data split. 23

3.2 Heatmap of Pearson correlation of log returns for all 93 stocks over the entire training split, clustered by sector. 24

3.3 Heatmap of Pearson correlation of log returns for the bottom 10% realised volatility days (LHS) and top 10% realised volatility days (RHS). 25

3.4 Plot of intraday realised volatility averaged over all stocks. 26

3.5 Autocorrelation functions of returns and squared returns in the training split. Returns are calculated within each trading day. The solid line shows the mean across stocks and the shaded region shows the interquartile range. 27

3.6 GraphTCN architecture. 28

4.1 Average learned graph components across all test windows. The top panel shows the final mixed graph used for message passing. The lower panels show the learned static graph and the mean context-dependent dynamic graph. Stocks are grouped by sector, rows represent target stocks, columns represent source stocks, and self-edges are excluded. 50

4.2 Example of a learnt graph on 18th March 2024 at 11:45am 51

4.3 Intraday entropy profile of learned graphs. 53

4.4 Correlation between the T850 target variable at the central grid point and surrounding grid points for each location. 58

4.5 Illustrative GraphTCN final-horizon forecasts for London and Singapore during the 2016 test period. From top to bottom, the panels show London at $H = 4$ (one day), London at $H = 120$ (30 days), Singapore at $H = 4$, and Singapore at $H = 120$. Dashed black lines show the observed central-grid T850 series, while solid blue lines show the GraphTCN forecasts. . 60

4.6 Mean dynamic incoming adjacency weights to the central Singapore grid point for December-February and June-August in the $H = 28$ model. . . 62

4.7 Incoming adjacency weights to the central Hong Kong grid point during Typhoon Vicente. The left panel shows the dynamic graph, while the right panel shows the final graph after mixing the dynamic and static components. 63

List of Tables

3.1 Description of features in the finance dataset. The target feature is Close. 21

3.2 Sector distribution of the stock universe. 22

3.3 Summary of train, validation and test splits. 22

3.4 Summary statistics for one-minute Close log returns in the training split. 26

3.5 Trainable parameters in GraphTCN, divided according to their optimiser parameter group. 41

3.6 Architectural settings of GraphTCN. 42

4.1 Forecasting results on the financial test set. Log MAE is reported in basis points. Win Rate and Sign Accuracy are reported as percentages, while both Pearson metrics are reported as correlation coefficients. The best result at each horizon is shown in bold and the second-best is highlighted. 44

4.2 Unweighted mean Log MAE across the five forecast horizons. 44

4.3 Log MAE relative to persistence for the least and most volatile stocks. Values are reported in basis points. Negative values, shown in bold, indicate lower error than persistence. 46

4.4 The 95% block-bootstrap confidence intervals for the Pearson correlation between predicted and realised cumulative log returns. Intervals are calculated using 10,000 bootstrap samples of complete trading days. Bounds are reported to three decimal places. Intervals that exclude zero, determined using the unrounded values, are shown in bold. 46

4.5 Token prediction accuracy for Token GraphTCN. Accuracy is reported as a percentage and Excess values are the improvement of the model versus predicting the most common k tokens from the training set. 47

4.6 Sensitivity of the frozen Kronos decoder to corruption of the token sequence. The uncorrupted reconstruction is shown in the first row. 48

4.7 Learned values of the GraphTCN graph gates. 49

4.8 Similarity between the learned graph components and the training-set absolute-correlation prior. 49

4.9 Forecasting results for alternative GraphTCN graph formulations. The best result at each horizon is shown in bold and the second-best is highlighted. Rankings are determined using the unrounded values. 54

4.10 Rank of GraphTCN after inclusion alongside the nine models evaluated by Shu *et al.* Results for each city are averaged across the 2016–2018 test years. Rank one denotes the best-performing model out of ten, and the final column reports the mean rank across cities. 57

4.11 Task-level comparison of GraphTCN with selected models across the 60 weather forecasting tasks. Comparator results are taken from the Sonnet study. 57

4.12 Learned GraphTCN gate values across the weather forecasting experiments. The gate α controls the mixture of the static and dynamic adjacency matrices, while β controls the mixture of the original temporal and graph-updated representations. 61

C.1 GraphTCN forecasting results for Hong Kong. 74

C.2 GraphTCN forecasting results for Cape Town. 74

C.3 GraphTCN forecasting results for London. 75

C.4 GraphTCN forecasting results for New York. 75

C.5 GraphTCN forecasting results for Singapore. 75

Chapter 1

Introduction

1.1 Motivation

Financial time-series forecasting is a challenging statistical learning problem. Asset prices are affected by a large number of interacting economic, market-wide and firm-specific factors, while predictive signal is generally weak relative to short-term market noise [30]. The underlying data-generating process is also non-stationary: volatility, liquidity and relationships between assets may change across time and market regimes. These characteristics make it difficult for a forecasting model to distinguish genuine structure from noise.

Furthermore, there is an entire field of financial literature which proposes that markets are efficient [10]. An efficient market means that the current price of a financial instrument reflects the aggregate knowledge of all market participants, and is therefore the best possible forecast of future price. Under this theory, prices follow a random walk, and the best forecasting model is therefore a simple persistence model:

$$\widehat{P}_{t+h} = P_t,$$

where P_t is the last observable price and h is the forecasting horizon. Although real financial markets need not follow an exact random walk, markets are efficient enough such that persistence provides a strong baseline model. Consequently, outperforming persistence provides a more demanding test than simply obtaining a small absolute forecasting error [10, 23].

Statistical models such as ARIMA and GARCH remain important benchmarks in financial forecasting. Their simple structure can be advantageous when the predictive signal is weak and the amount of data available is limited. Evidence from the broader forecasting literature suggests that machine-learning methods do not consistently outperform these simple approaches in the financial domain [26]. These conventional models are commonly fitted independently to each series, while multivariate statistical methods such as vector autoregression typically represent cross-series dependence through fixed linear relationships. However, financial assets do not evolve independently, and relationships between stocks are generally dynamic and highly non linear. Multivariate neural forecasting can therefore offer a potential advantage by allowing nonlinear information observed in related assets to inform each forecast. However, naive mixing across all series may increase model complexity and encourage overfitting [41], which is generally the main cause of the poor performance of neural models in the financial domain. The forecasting problem is therefore not only to model the temporal behaviour of each asset, but also to determine when and how information should be exchanged between assets in a way that keeps the model as parsimonious as possible.

Geometric deep learning provides a natural framework for introducing this relational structure [5]. A multivariate time series can be represented as a graph in which each series is a node and weighted edges determine how information is propagated between nodes. For financial forecasting, the nodes may represent individual assets, while an edge represents a learned message-passing weight. Spatio-temporal graph architectures can then separate two modelling problems: *intra-series learning*, which captures the temporal dynamics of each individual node, and *inter-series learning*, which captures relationships between nodes.

Dynamic graph learning extends this idea by allowing the graph to depend on the current input context. Instead of learning one fixed graph over the training period, a dynamic graph can represent relationships that strengthen, weaken or change direction across time. Nevertheless, a purely dynamic graph may be noisy and difficult to estimate from a short context window [38]. A combination of a static and adaptive graph is therefore attractive: a static graph can encode persistent relationships, while a dynamic graph can modify those relationships in response to the current market state.

Motivated by this, the principal model developed in this dissertation is *GraphTCN*, a geometric extension of ModernTCN [25], which is a state of the art deep learning model for time series forecasting. ModernTCN is used to learn temporal representations independently for each series, after which a graph-learning module enables cross-series learning. GraphTCN combines a trainable static graph with a dynamic graph that is conditioned on the output of the ModernTCN layer. Learned gates control both the balance between

the static and dynamic graph components and the contribution of spatial information relative to the original temporal representation. This design preserves a strong temporal forecasting backbone while making cross-series interaction adaptive. A key benefit of this architecture is that it exposes the trained graphs, giving insight into the relationships the model has learnt and therefore gives a level of interpretability which most other deep learning based time series architectures do not.

The primary empirical study is based on a financial dataset consisting of one-minute price observations for 93 stocks over a 1 year period. Models receive a 60-minute context and produce forecasts at horizons of 1, 5, 15, 30 and 60 minutes. GraphTCN is evaluated against persistence, statistical forecasting models, ModernTCN, and *Kronos* [32], a zero-shot foundational time series model which has been pre trained on a large corpus of financial data.

We then evaluate GraphTCN on a secondary spatial weather forecasting benchmark introduced by Shu *et al* [33]. This benchmark comprises of five city-specific datasets, each containing weather related measurements from nine locations arranged in a 3×3 spatial grid, with temperature at the central grid point as the forecasting target. Weather data contains clearer temporal structure, including seasonality, and measurements at neighbouring grid points are linked by physical atmospheric processes. The weather task therefore provides a setting in which both the temporal model and the graph component have stronger, more interpretable structure to exploit.

1.2 Research Objectives

This dissertation addresses four research questions:

1. Can GraphTCN improve multi-horizon intraday equity forecasting relative to persistence and established statistical, neural, and foundation-model baselines?
2. Does augmenting ModernTCN with a graph-learning module for cross-series learning improve forecasting performance over just using ModernTCN to learn both intra and inter series relationships?
3. Can a dynamic graph adapt to changes in the observed time series and provide additional forecasting value beyond a static graph structure?
4. Can GraphTCN generalise to a less noisy weather forecasting problem, where temporal patterns are clearer and physical relationships between neighbouring locations provide a stronger basis for graph-based modelling?

1.3 Report Overview

Chapter 2 provides the background required to understand the forecasting problem and the proposed architecture. We first give an overview of the multivariate time series forecasting problem, covering the major statistical and neural approaches. We then give a theoretical introduction to geometric deep learning, introducing the key ideas that underlay the field. We then extend geometric deep learning to the multivariate time series forecasting setting. Finally we introduce two non geometric time series models; ModernTCN, which forms a key part of our GraphTCN architecture, and the Kronos financial foundation model, which takes a different approach to forecasting by first tokenizing the continuous financial data into discrete tokens, turning the forecasting problem into a next token prediction task.

Chapter 3 describes the experimental methodology. It introduces the financial dataset in detail, including key statistical properties of the data. We then present the GraphTCN architecture in detail. Next, we formalise the forecasting task, describe the baseline models, together with the training procedure, hyperparameter-selection process, and evaluation metrics.

Chapter 4 presents and discusses the experimental results. We first evaluate the models on the financial dataset, including GraphTCN's forecasting performance, learned graph structure, ablation results, and behaviour across stocks and market conditions. We then briefly introduce the secondary weather dataset and formalise the weather forecasting task, and then evaluate GraphTCN on the weather benchmark across multiple cities, forecasting horizons, and test years.

Chapter 5 concludes the dissertation by summarising the main findings. It discusses the limitations of the experimental design, before outlining directions for future work in graph-based forecasting.

Chapter 2

Background and Related Work

2.1 Time-Series Forecasting

Time-series forecasting aims to estimate future observations from information available up to a forecast origin. Given a look-back window of length L, a forecasting model receives an input sequence

$$\mathbf{X}_t = [\mathbf{x}_{t-L+1}, \ldots, \mathbf{x}_t] \in \mathbb{R}^{L \times C},$$

where C denotes the number of features per observation. The model produces forecasts for the following H time steps,

$$\widehat{\mathbf{Y}}_t = f_\theta(\mathbf{X}_t) \in \mathbb{R}^{H \times M},$$

where M is the number of forecast variables and f_θ is a model parameterised by θ. Forecasts may be generated auto-regressively, by repeatedly applying a one-step model, or directly, by predicting the complete H-step output sequence in one forward pass. Auto-regressive forecasting uses a compact output model but can accumulate error as earlier predictions are fed back as inputs. Direct forecasting avoids this error propagation, although it requires the model to learn a separate mapping for multiple future positions.

A *univariate* forecasting problem is one which uses the history of a single series to forecast that same series, such that $C = M = 1$. A *multivariable* forecasting problem uses multiple endogenous or exogenous input variables to forecast one designated target, such

that $C > 1$ and $M = 1$. By contrast, a *multivariate* forecasting problem jointly predicts multiple output series, such that $C > 1$ and $M > 1$. In the latter case, the model may forecast all observed series simultaneously and can therefore learn both temporal structure within each series and dependencies between series. In this thesis, the finance dataset represents a *multivariate* forecasting problem since we are trying to forecast prices for all assets simultaneously. The weather benchmark represents a *multivariable* forecasting problem since there is only one target series. These datasets will be discussed in more detail later.

The simplest forecasting methods model each series independently. Autoregressive integrated moving-average models, or ARIMA, represent the current value of a series using a linear combination of its previous values and previous forecast errors, after differencing the series where necessary to reduce non-stationarity [34]. An ARIMA model can be expressed using the lag operator B as

$$\phi(B)(1 - B)^d y_t = c + \theta(B)\varepsilon_t,$$

where d is the order of differencing, $\phi(B)$ describes the autoregressive terms, and $\theta(B)$ describes the moving-average terms. ARIMA provides a transparent and parsimonious representation of linear temporal dependence, but its standard formulation is univariate and assumes that the fitted relationships remain stable over time.

Financial and economic series often exhibit periods of changing variance that are not captured by a constant-noise ARIMA model. ARCH models and their generalised form, GARCH, address this by modelling the conditional variance of the residual process [4, 9]. A GARCH(p, q) model specifies

$$\sigma_t^2 = \omega + \sum_{i=1}^{q} \alpha_i \varepsilon_{t-i}^2 + \sum_{j=1}^{p} \beta_j \sigma_{t-j}^2,$$

where σ_t^2 is the conditional variance at time t.

A direct linear extension to multiple series is provided by vector autoregression. For an N-dimensional vector $\mathbf{y}_t$, a VAR model of order p takes the form

$$\mathbf{y}_t = \mathbf{c} + \sum_{i=1}^{p} \mathbf{A}_i \mathbf{y}_{t-i} + \boldsymbol{\varepsilon}_t.$$

The coefficient matrices $\mathbf{A}_i$ allow the lagged values of every series to contribute to every forecast variable. VAR therefore models cross-series dependencies, but assumes that these relationships are linear and constant throughout the fitted period. Its parameter count also increases rapidly with the number of series and lagged observations, making unrestricted estimation difficult in high-dimensional settings.

Neural forecasting methods replace these fixed linear relationships with learned nonlinear transformations. Recurrent neural networks process observations sequentially while maintaining a hidden state,

$$\mathbf{h}_t = \phi\left(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}\right),$$

where $\mathbf{h}_t$ is intended to summarise the information observed up to time t. In principle, this recurrent state allows the model to represent nonlinear temporal dependencies of arbitrary length. In practice, training a conventional recurrent network through many time steps is difficult because gradients may repeatedly shrink or grow during backpropagation through time. This produces the well-known vanishing- and exploding-gradient problems, which limit the ability of a standard recurrent network to learn long-range dependencies reliably [3].

Long short-term memory networks address this limitation by introducing a persistent cell state and a set of learned gates that control the flow of information [14]. The input gate determines how much new information enters the cell, the forget gate determines how much existing information is retained, and the output gate controls how the state contributes to the hidden representation. Gated recurrent units provide a related but more compact formulation by combining some of these gating operations [6]. Both architectures improve the ability of recurrent models to preserve information over long sequences and have been widely applied to forecasting. Nevertheless, LSTMs and GRUs retain the sequential computation of a recurrent network, which limits parallelisation during training and can make long input windows computationally expensive.

Transformers provide an alternative by replacing recurrence with self-attention [36]. Given query, key and value representations, scaled dot-product attention is defined as

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}.$$

This operation allows every input position to interact directly with every other position, and the complete input sequence can be processed in parallel. Causal masking can be used

to ensure that each temporal representation depends only on current and previous observations. The flexibility of attention also introduces difficulties. Standard self-attention has quadratic computational and memory complexity in the sequence length, which can become expensive for high-frequency or long-horizon forecasting. Modern Transformer based architectures such as *PatchTST* attempt to solve some of these problems by using patching [29]. Instead of feeding individual time steps into the Transformer one by one, patching groups adjacent points together, which allows the model to better capture local semantics and reduce the memory and computational costs.

Temporal convolutional networks provide yet another alternative to recurrent modelling. A TCN uses causal one-dimensional convolutions so that an output at time t depends only on observations at or before t. Dilated convolutions expand the receptive field by skipping input positions at a fixed interval,

$$(\mathbf{x} *_d \mathbf{k})(t) = \sum_{i=0}^{K-1} k_i \, x_{t-d_i},$$

where K is the kernel size and d is the dilation factor. By stacking causal convolutions with increasing dilation, a TCN can represent long temporal dependencies while retaining parallel computation across the sequence. Residual connections further improve optimisation by providing short gradient paths through deep convolutional stacks.

Compared with recurrent networks, TCNs offer stable parallel training and a receptive field that can be calculated directly from the kernel sizes, dilations and model depth. Compared with Transformers, their computational cost generally scales linearly with the input length and their convolutional structure introduces a stronger prior for local temporal patterns. However, the receptive field is fixed by the architecture rather than selected dynamically for each input, and long contexts may require deep networks, large dilation factors or large kernels. An example of a recent TCN based architecture is *Modern TCN* [25], which plays a key role in our *GraphTCN* architecture and will therefore be discussed in detail later.

The methods reviewed above provide increasingly flexible ways to model temporal dependence. When multiple series are observed together, however, a separate question remains: how should information be exchanged between them? Combining all variables within a single unrestricted representation can make it difficult to distinguish useful relationships from weak or unstable dependencies, and can also lead to overfitting [41]. Geometric deep learning provides a structured alternative by representing each series as a node in a graph and modelling temporal dynamics and cross-series interactions as distinct components.

2.2 Geometric Deep Learning

Many datasets contain relationships that cannot be represented naturally by a regular sequence or grid. Examples include molecular bonds, transport networks and social interactions. Geometric deep learning extends neural methods to such non-Euclidean domains by incorporating the structure of the data into the model's inductive bias [5]. This dissertation focuses on graph neural networks, which represent a collection of entities as nodes and their relationships as edges.

A graph is denoted by

$$G = (V, E),$$

where $V = \{v_1, \ldots, v_N\}$ is a set of N nodes and E is a set of edges connecting them. Each node v may have an associated feature vector $\mathbf{x}_v \in \mathbb{R}^d$, and the complete node-feature matrix is written as

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_N^\top \end{bmatrix} \in \mathbb{R}^{N \times d}$$

The graph structure can be represented by an adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$. For a binary graph, $A_{vu} = 1$ indicates that an edge connects source node u to target node v. For a weighted graph, A_{vu} determines the strength with which information from u contributes to v. The graph may be undirected, in which case $\mathbf{A}$ is symmetric, or directed, in which case A_{vu} and A_{uv} may differ.

2.2.1 Graphs and Message Passing

The general mechanism used by graph neural networks is known as *message passing* [5]. Let $\mathbf{h}_v^{(0)} = \mathbf{x}_v$ denote the initial representation of node v - that is to say that our initial node embedding is simply the feature vector associated with that node (and 1 if there is no feature vector). At layer ℓ, each neighbouring source node u constructs a message for target node v:

$$\mathbf{m}_{v \leftarrow u}^{(\ell)} = f^{(\ell)} \left(\mathbf{h}_v^{(\ell)}, \mathbf{h}_u^{(\ell)}, A_{vu} \right),$$

where $f^{(\ell)}$ is a shared learnable function. The messages received by v from all neighbours are then combined using an aggregation function,

$$\mathbf{m}_v^{(\ell)} = \mathrm{AGG}\left(\left\{\mathbf{m}_{v\leftarrow u}^{(\ell)} : u \in \mathcal{N}(v)\right\}\right),$$

where $\mathcal{N}(v)$ denotes the neighbourhood of v, which is the set of nodes u that have non zero edge E connecting them to v. The node representation at v is then updated as

$$\mathbf{h}_v^{(\ell+1)} = g^{(\ell)}\left(\mathbf{h}_v^{(\ell)}, \mathbf{m}_v^{(\ell)}\right),$$

where $g^{(\ell)}$ is another learnable function. The update normally retains information from the target node itself, either through an explicit self-loop, concatenation or a residual connection.

This formulation separates two operations. The message function determines what information a source node transmits, while the aggregation function determines how the target combines messages from its neighbours. After one layer, a node representation contains information from its immediate neighbourhood. Stacking K layers expands the receptive field to nodes up to K edges away. However, repeatedly averaging neighbouring representations can eventually make node embeddings overly similar, a phenomenon known as over-smoothing.

A Graph Convolutional Network (GCN) provides a simple example of message passing [19]. A GCN updates the representation of target node v by aggregating transformed representations from its neighbouring source nodes u,

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\sum_{u\in\mathcal{N}(v)\cup\{v\}} \overline{A}_{v,u}\mathbf{W}^{(\ell)}\mathbf{h}_u^{(\ell)}\right),$$

where $\overline{\mathbf{A}}$ is a normalised adjacency matrix, $\mathbf{W}^{(\ell)}$ is a learned feature transformation, and σ is a nonlinear activation function. The adjacency weight $\overline{A}_{v,u}$ determines the contribution of source node u to target node v. A conventional GCN therefore learns how node features should be transformed, but assumes that the graph structure has already been supplied.

Graph Attention Networks (GATs) allow the relative importance of neighbouring source nodes to be learned from their representations [37]. The representation of target node v is updated according to

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{v,u} \mathbf{W}^{(\ell)} \mathbf{h}_u^{(\ell)}\right),$$

where $\alpha_{v,u}$ is a learned attention weight which measures the importance of source node u to target node v. GATs can therefore assign different importance to different neighbours. However, the neighbourhood $\mathcal{N}(v)$ is still normally defined by a supplied graph.

2.2.2 Static, Learned, and Dynamic Graphs

In some problems, the graph structure is supplied by domain knowledge. Road segments may be connected according to the transport network, atoms according to chemical bonds, or spatial locations according to physical proximity. This information can then be used to construct the graph adjacency which will be used in the aggregation operation defined in the previous section.

However, in some settings a suitable graph may not be known *a priori*. For example, when forecasting stock prices, it is unclear exactly what the relationship between different stocks should be, and therefore unclear how our graph should be defined. In these cases, the graphs edge weights can instead be learned jointly with the forecasting model. A *learned static graph* is optimised from the training data but remains fixed across all inputs. It can be parameterised directly through trainable edge weights or indirectly through learned node embeddings.

A static graph is suitable when relationships are expected to remain broadly stable, but it cannot represent changes between observations. A *dynamic graph*, A_t, instead derives its adjacency matrix from the current input, $\mathbf{A}_t = \Gamma_\alpha(\mathbf{H}_t)$, where $\mathbf{H}_t$ contains the node representations at time t, and Γ_α is a learnable function parametrised by α. Different inputs can therefore produce different message-passing weights. This allows the model to strengthen or weaken relationships according to the state currently being observed rather than assuming that one graph is appropriate throughout the dataset.

The additional flexibility of a dynamic graph comes with a disadvantage: the graph must be estimated from a finite and potentially noisy input, and this instability can lead to convergence issues in training [38]. A learned static graph is more stable but cannot adapt. Hybrid architectures therefore combine a static graph to represent relationships that hold across the training data and a dynamic graph to capture sample-specific changes.

When node features evolve over time, for example if they are time series, graph learning

must be combined with a temporal model. The temporal model first summarises the history of each node, after which graph-based message passing exchanges information between the resulting representations. The next section reviews how this temporal and relational decomposition has been used in geometric approaches to multivariate time-series forecasting.

2.3 Geometric Deep Learning for Time Series

A multivariate time series can be represented as a sequence of graph node features. Each series is treated as a node in the graph, while the observations associated with that series form its time-varying node features. At time t, the complete observation can therefore be written as $\mathbf{X}_t \in \mathbb{R}^{N \times C}$, where N is the number of series and C is the number of features observed for each node.

Graph-based forecasting models typically separate the learning of temporal and cross-series dependencies, as illustrated in Figure 2.1. A temporal module extracts patterns from the history of each node, while a spatial module uses graph-based message passing to exchange information between nodes. In a simplified form, for a given input time series $\mathbf{x}^{(n)}$ and look back window L, the temporal model (which can be any of the neural time series architectures discussed) learns an embedding, $h^{(n)}$:

$$\mathbf{h}^{(n)} = \mathcal{T}_\theta \left(\mathbf{x}^{(n)}_{t-L:t} \right),$$

where $\mathcal{T}_\theta$ is the temporal model parametrised by θ. The embeddings for different series are then aggregated and mixed conditional on the graph. This gives the updated node embedding:

$$\widetilde{\mathbf{h}}^{(n)} = g_\phi(\mathbf{h}^{(1:N)}; \mathbf{A}),$$

where A is the graph adjacency, g is a generic aggregation and spatial message passing function parametrised by ϕ and $\widetilde{\mathbf{h}}^{(n)}$ is the updated embedding for node n, which recall represents the nth time series. The term *spatial* is commonly used for this cross-series operation even when the nodes do not represent physical locations. The temporal and spatial representations are then used to predict future values.

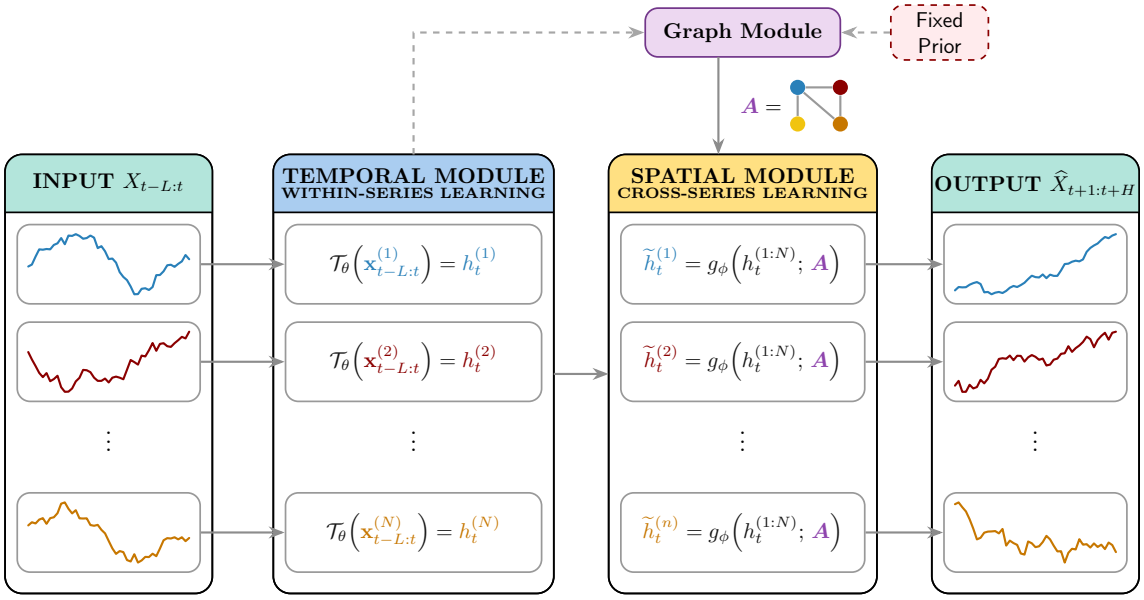


Figure 2.1: Generic structure of a spatio-temporal graph forecasting model. A temporal module extracts patterns within each series, while a graph-based spatial module exchanges information between series according to a supplied or learned adjacency matrix.

The majority of geometric deep learning based time series models in the literature focus on traffic forecasting. One of the first models was the Diffusion Convolutional Recurrent Neural Network (DCRNN), proposed by Li *et al* [22]. DCRNN uses a graph convolution style spatial mixing module in combination with a RNN temporal module. Traffic sensors at different locations in the road network are represented as nodes in the graph, and the graph edge weights are derived from the distance between the sensors. The graph is therefore supplied by domain knowledge rather than learnt, and remains fixed across forecasting examples.

This assumption is unsuitable for many multivariate datasets in which no reliable graph is known in advance. Wu *et al* address this limitation with MTGNN, which learns the graph directly from the data [38]. The graph learner begins with two trainable node-embedding matrices $\mathbf{E}_1, \mathbf{E}_2 \in \mathbb{R}^{N \times d_e}$. They are transformed as

$$\mathbf{M}_1 = \tanh\left(\mathbf{E}_1 \mathbf{W}_1\right), \qquad \mathbf{M}_2 = \tanh\left(\mathbf{E}_2 \mathbf{W}_2\right),$$

where $\mathbf{W}_1$ and $\mathbf{W}_2$ are learnable weight matrices. $\mathbf{M}_1$ and $\mathbf{M}_2$ are then used to construct the adjacency matrix

$$\mathbf{A} = \mathrm{TopK}\left[\mathrm{ReLU}\left(\tanh\left(\mathbf{M}_1 \mathbf{M}_2^\top - \mathbf{M}_2 \mathbf{M}_1^\top\right)\right)\right].$$

The difference term $\mathbf{M}_1\mathbf{M}_2^\top - \mathbf{M}_2\mathbf{M}_1^\top$ allows the weight from node v to node u to differ from the weight in the reverse direction. The ReLU removes negative weights, while the row wise top-k operation retains only the strongest learned relationships and produces a sparse graph. Temporal-convolution and graph-convolution modules are then interleaved to model within-series and cross-series dependencies, respectively.

DCRNN and MTGNN illustrate two important approaches to graph-based time-series forecasting. In the former, the graph is supplied using known physical structure; in the latter, a global graph is learned from the data. In both cases, the same adjacency structure is used across forecasting windows. Li *et al* addressed this limitation in the Dynamic Graph Convolutional Recurrent Network (DGCRN), another architecture developed for traffic forecasting [20]. DGCRN combines a predefined static road graph with a dynamic graph generated at each time step. The static graph, A^{static}, is constructed from distances between locations in the road network whilst the dynamic graph is conditioned on the traffic state currently being processed.

At time t, DGCRN concatenates the current traffic measurements $\mathbf{V}_t$, time-of-day information $\mathbf{T}_t$, and the previous temporal hidden state $\mathbf{H}_{t-1}$:

$$I_t = \mathbf{V}_t \parallel \mathbf{T}_t \parallel \mathbf{H}_{t-1}.$$

I_t is then used to build the dynamic graph, $A_t^{dynamic}$, where the exact details have been left out as they are not particularly important to this discussion. Finally, the dynamic and static graphs are combined using learnt hyperparameters α and γ:

$$\widehat{A_t} \approx \alpha A^{static} + \gamma A_t^{dynamic},$$

and $\widehat{A_t}$ is then used in the spatial mixing operations. The parameters α and γ allow the model to learn how much of each graph to use. This allows the model to rely more heavily on the pre defined static graph earlier in training while the dynamic graph structure is being learnt.

The architectures discussed in this section describe a progression from a supplied fixed graph, to a learned global graph, and finally to the joint use of static and context-dependent graph structure. The latter provides an important precedent for the architecture developed in this dissertation, which similarly combines a stable graph component with a dynamic graph inferred from the representations of the current input window.

2.4 ModernTCN

The preceding discussion has focused on geometric deep-learning approaches to time-series forecasting, in which relationships between series are represented through a graph. As discussed, these use a temporal module to extract temporal information from the time series. For our GraphTCN architecture, we use *ModernTCN* as our temporal backbone, which is a temporal convolution based model.

ModernTCN was proposed by Luo *et al* as a pure convolutional architecture for time-series analysis [25]. The model is intended to retain the computational efficiency of convolution while increasing its effective receptive field and allowing both temporal and inter-variable dependencies to be learned. Since we make extensive use of ModernTCN in our GraphTCN architecture, we discuss it in detail here.

2.4.1 Patch Embedding

ModernTCN first starts with a patch embedding layer. Consider a batch B containing N input series observed over L time steps. The input is written as $\mathbf{X} \in \mathbb{R}^{B \times N \times L}$. Rather than applying the convolutional operation directly to all L individual observations, ModernTCN first divides each variable into short, potentially overlapping patches. These patches are then projected into an embedding space before convolutional operations are applied. A one-dimensional convolution (i.e. a linear projection) performs the patching and embedding operations simultaneously:

$$\mathbf{E} = \text{PatchEmbed}(\mathbf{X}) \in \mathbb{R}^{B \times N \times D \times P},$$

where D is the learned feature dimension and P is the number of patch positions. The same patch-embedding operation is applied to each of the N series. The resulting embedding matrix now treats the time series as a sequence of P patches, each consisting of N series, where each series is now represented in a D-dimensional embedding space.

Patching serves two purposes. First, each input presented to the model will contain information from several neighbouring observations, rather than representing a single time step. This allows the model to learn local temporal features more easily. Second, the temporal axis is shortened from L observations to P patches, reducing the number of positions processed by subsequent layers.

2.4.2 Convolutions

Next, the learned patch embedding is passed into the ModernTCN convolution block. This block consists of a series of convolution operations applied sequentially over each dimension of the embedding matrix. These convolution operations use a combination of large and small convolutional kernels, giving the ability to model long and short range dependencies.

The first layer in this block, **DWConv**, applies a convolutional filter over the patch time dimension P. For a given batch b, series n and embedded dimension d, the layer takes $E[b, n, d, :] \in \mathbb{R}^P$ and applies a convolutional filter over the resulting vector. This stage uses a large kernel, K^{large}, who's width is usually set to 51 by default. The combination of large kernel and patching provides the model with a large Effective Receptive Field (ERF). This allows the model to capture long-range dependencies as effectively as the global mixing of self-attention mechanisms, but in linear time.

The temporal transformation is

$$\mathbf{E}^{'} = \text{DWConv}(\text{LayerNorm}(\mathbf{E})).$$

The cross-time relationships are now encoded, but the distinct series (N) and their embedded features (D) have not yet interacted.

The next layer in this block, **ConvFFN1**, applies a smaller filter over the embedded dimension, D. For a given batch b, series n and patch time position p, the layer receives a vector $E^{'}[b, n, :, p] \in \mathbb{R}^D$. ConvFFN1 first maps this vector through a bottleneck structure defined by $\mathbb{R}^D \to \mathbb{R}^{rD} \to \mathbb{R}^D$, where r is an expansion ratio used to project the features into a higher-dimensional space before compressing them back. This operation is carried out by a feed forward network: $\mathbf{G} = FFN_\theta(\mathbf{E}^{'})$. In practise, the authors recommend $r = 1$ for most datasets to avoid over fitting. A convolution filter is then applied to the compressed vector over the embedding dimension, d :

$$\mathbf{G}^{'} = \text{ConvFFN1}(\text{LayerNorm}(\mathbf{G})).$$

This allows the model to learn relationships between embedding dimensions within each series.

The final layer, **ConvFFN2**, mixes across series using the same small kernel used in CovFFN1. For a given batch b, embedding dimension d and patch time p, we take the

vector $G^{'}[b,:,d,p] \in \mathbb{R}^{\mathbb{N}}$ and again pass it through a bottleneck structure with the same expansion ratio, r, as the previous layer. The bottleneck is now defined by $\mathbb{R}^N \rightarrow \mathbb{R}^{rN} \rightarrow \mathbb{R}^N$, and is again implemented by a feed forward network: $\mathbf{Z} = FFN_{\phi}(\mathbf{G}^{'})$. We then apply the same small convolutional kernel, but now over the series dimension:

$$\mathbf{Z}^{'} = \text{ConvFFN2}(\text{LayerNorm}(\mathbf{Z})).$$

This is therefore the layer in the architecture that enables the model to learn cross series relationships. The sequence for a full ModernTCN block is therefore given by:

$$E = PatchEmbed(X) \rightarrow \text{DWConv} \rightarrow \text{ConvFFN1} \rightarrow \text{ConvFFN2} \rightarrow Z^{'}$$

These 3 convolution operations represent 1 ModernTCN block. Blocks can be stacked, but the authors note that one block is enough for most datasets, and that stacking multiple blocks usually leads to over fitting. The output of the final block is then passed to a task specific output head.

ModernTCN provides a strong and computationally efficient foundation for learning temporal representations. Its original study reported state of the art performance across five major time-series analysis tasks: long and short term forecasting, imputation, classification, and anomaly detection, while retaining the efficiency advantages of a purely convolutional architecture. Subsequent evaluations have also identified ModernTCN as one of the stronger deep learning based time series approaches for financial data. For example, a recent study compared 9 different deep learning architectures on a financial time series forecasting benchmark which consisted of more than 900 different series [31]. ModernTCN achieved the highest overall rank across all models and was the best model in 75% of the forecasting tasks. We therefore believe that ModernTCN is a suitable temporal backbone for our architecture.

2.5 Kronos

We now consider a contrasting approach to financial time series forecasting through *Kronos* [32], a pretrained financial time-series foundation model that learns forecasting behaviour from large-scale pretraining. Its central idea is to treat a sequence of continuous market observations as a discrete language. Rather than forecasting future values directly through regression, Kronos first converts each multivariate observation into a discrete to-

ken and then learns to predict the resulting token sequence autoregressively.

The model follows a two-stage framework: a Transformer-based tokenizer is first trained to encode and reconstruct continuous price observations. Once this has been trained, the Tokenizer encoder converts the continuous price data into discrete tokens. A second decoder-only Transformer is then trained to forecast the next token [32], after which the pre trained decoder converts the predicted token back to continouous price. Both stages are pretrained on more than 12 billion examples of price data over 45 global exchanges. We cover each stage separately.

2.5.1 Tokenizer

Let $\mathbf{x}_t \in \mathbb{R}^D$ denote the observation at time t, where Kronos uses $D = 6$. A Transformer encoder maps the continuous input into a latent representation $\boldsymbol{\xi}_t$. This representation is then discretised using a method called Binary Spherical Quantisation (BSQ) [40].

BSQ first projects the latent representation into a k-dimensional space,

$$\mathbf{v}_t = W_q \boldsymbol{\xi}_t,$$

and normalises it onto the unit hypersphere:

$$\mathbf{u}_t = \frac{\mathbf{v}_t}{\|\mathbf{v}_t\|_2}.$$

Each dimension is then binarised according to its sign:

$$\widehat{\mathbf{u}}_t = \frac{1}{\sqrt{k}} \operatorname{sign}(\mathbf{u}_t)$$

so that

$$\widehat{\mathbf{u}}_t \in \left\{ -\frac{1}{\sqrt{k}}, +\frac{1}{\sqrt{k}} \right\}^k.$$

Each coordinate of $\widehat{\mathbf{u}}_t$ falls on one corner of a hypercube projected onto the unit sphere. Since $\widehat{\mathbf{u}}_t \in \mathbb{R}^k$, the number of different possible tokens is 2^k. The discrete tokens are then passed through a temporal decoder which reconstructs the original input.

Kronos uses a 20-bit code (i.e. $k = 20$) , which would produce a vocabulary of $2^{20} =$ 1,048,576 complete tokens. Predicting over such a large vocabulary would require an impractically large output layer in the forecasting model. Each token is therefore divided into two 10-bit components, denoted s_1 and s_2. Each s_i therefore has a vocabulary of only $2^{10} =$ 1,024 entries. The first component, s_1, also called the **coarse token**, is trained to capture the main structure of the continuous input data. The second component, s_2, called the **fine token**, acts as a fine tuner that provides the residual information needed for a more accurate reconstruction. This hierarchy is imposed directly through the tokenizer loss function, $\mathcal{L}_{\text{tokenizer}} \approx \mathcal{L}_{\text{coarse}} + \mathcal{L}_{\text{fine}}$, where $\mathcal{L}_{\text{coarse}}$ is the reconstruction loss using just the coarse token and $\mathcal{L}_{\text{fine}}$ is the reconstruction loss from using both the coarse and fine token.

The motivation for this representation is twofold. First, assigning similar observations to the same token can suppress small, potentially noisy variations and limit the influence of extreme values. Second, it transforms forecasting over an effectively infinite continuous space into a next token prediction task, which the authors argue is an easier forecasting problem. The cost is that tokenisation is lossy.

2.5.2 Autoregressive Token Forecasting

Once the tokenizer has been trained, forecasting becomes a next-token prediction problem. For a historical token sequence $\mathbf{s}_{1:T}$, Kronos models the future distribution as

$$p(\mathbf{s}_{T+1:T+H} \mid \mathbf{s}_{1:T}) = \prod_{h=1}^{H} p(\mathbf{s}_{T+h} \mid \mathbf{s}_{1:T+h-1}) \,.$$

A causal decoder-only Transformer processes the historical s_1 and s_2 tokens. At each future time step, the coarse token is predicted first, after which the fine token is predicted conditionally:

$$p(\mathbf{s}_t \mid \mathbf{s}_{<t}) = p(s_{1,t} \mid \mathbf{s}_{<t})\, p(s_{2,t} \mid \mathbf{s}_{<t}, s_{1,t}).$$

The corresponding objective is the sum of the cross-entropy losses for the two token streams:

$$\mathcal{L} = -\sum_{t} \left[\log p(s_{1,t} \mid \mathbf{s}_{<t}) + \log p(s_{2,t} \mid \mathbf{s}_{<t}, s_{1,t}) \right] .$$

During inference, we obtain a prediction by sampling the next token from the distribution produced by the forecasting model. An s_1 token is sampled first, followed by its conditional s_2 token; the pair is appended to the context and the process is repeated for the required forecast length. The generated token sequence is finally passed through the frozen tokenizer decoder to recover continuous observations. Multiple sampled trajectories can also be decoded and averaged to form the final forecast.

We use Kronos as a baseline model in our financial prediction task, and motivated by its tokenization approach, we also train a tokenized version of our GraphTCN architecture using the Kronos tokenizer. The details of this architecture will be discussed in the next chapter.

2.6 Summary

This chapter first introduced the time series forecasting problem and covered some of the main approaches used in the literature. The performance of simple statistical models in the financial time series forecasting literature, and in particular the persistence model, motivates their use as baselines in our experiments. We then established the conceptual and architectural foundations for GraphTCN. We outlined the key ideas behind geometric deep learning, and covered a few of the main approaches in using geometric deep learning for time series forecasting. We introduced some of the main ways geometric time series models go about learning graphs for message passing. Since the relationships between stocks are expected to be dynamic, we will use some of the ideas from the DGCRN architecture in our graph learning module.

We then introduced ModernTCN, and its strong performance across a range of forecasting tasks, including financial forecasting, motivates its use as our temporal module. Using the standard, non graph based version of ModernTCN as a baseline also gives us a direct comparison with our architecture, and helps us to understand if geometric deep learning is a sensible approach to learn cross series relationships. Finally we presented an alternative way of forecasting financial time series using the discretisation approach of the zero shot foundation model Kronos. The strong performance of Kronos in the papers experiments suggests that tokenizing may improve the ability of a model to forecast noisy financial data, and we will therefore test this hypothesis on our dataset by using Kronos as a baseline and also training a tokenized version of GraphTCN.

Chapter 3

Methodology

3.1 Dataset

We now introduce the primary financial dataset used in this thesis. It has been provided by our IXN partner, *Autonomous-Fox Laboratories*. The dataset consists of price and volume data for 93 large cap developed market equities over the year 2024. The data is sampled at 1 minute intervals, and since a trading day runs from 09:30 to 16:00, this gives 390 observations per stock per day. Each observation is a 5 dimensional feature vector: $\mathbf{x}_{n,t} \in \mathbb{R}^5$, for stock n at time t. The 5 recorded features are Open, High, Low and Close prices and trading Volume, also known as OHLCV. The goal of this thesis is to use all 5 features to predict the Close price for every stock at 5 forecasting horizons: 1, 5, 15, 30 and 60 minutes. This is therefore a *multivariate* forecasting problem. The interpretation of each feature are summarised in Table 3.1.

Feature	**Description**
Open	The first traded price during minute t.
High	The highest traded price during minute t.
Low	The lowest traded price during minute t.
Close	The final traded price during minute t.
Volume	The total number of shares traded during minute t.

Table 3.1: Description of features in the finance dataset. The target feature is Close.

The stocks in the dataset span a range of economic sectors, which is summarised in Table 3.2. The sector information provides an economically meaningful reference against which the learned graph relationships can later be interpreted - we should logically expect stocks in the same sector to influence each other more than stocks in different sectors.

The sector labels are used only for descriptive analysis and visualisation and are not used as input features.

Sector	Number of stocks	Percentage of universe
Basic Materials	8	8.6%
Communication Services	9	9.7%
Consumer Cyclical	9	9.7%
Consumer Defensive	10	10.8%
Energy	9	9.7%
Financial Services	9	9.7%
Healthcare	10	10.8%
Industrials	10	10.8%
Technology	10	10.8%
Utilities	9	9.7%
Total	**93**	**100.0%**

Table 3.2: Sector distribution of the stock universe.

3.1.1 Train-Validation-Test Split

The data were partitioned chronologically rather than through random sampling. The splits are summarised in Table 3.3.

Split	Start date	End date	No. days in split	Forecast windows
Train	2024-01-02	2024-08-30	167	3,173
Validation	2024-09-03	2024-09-30	20	380
Test	2024-10-01	2024-12-31	62	1,178

Table 3.3: Summary of train, validation and test splits.

Under the forecasting setup introduced later in section 3.3.1, each complete day produces 19 forecast origins. Since the goal of this thesis is to forecast *intraday* prices, every context and its associated targets remain within a single trading day, so overnight price changes are excluded.

The construction of the splits preserve the temporal order of the observations and ensure that data from later periods do not enter the training examples. It also creates a realistic distributional setting in which the model is fit on earlier market conditions and subsequently applied to later periods.

To assess whether the three partitions represent substantially different market conditions,

Figure 3.1 compares their daily volatility distributions. Volatility is calculated from one-minute Close log returns. For stock n, minute t, and day d, the return is

$$r_{d,t,n} = \log C_{d,t,n} - \log C_{d,t-1,n}.$$

A realised-volatility estimate is first calculated independently for each equity:

$$RV_{d,n} = \sqrt{\sum_{t} r_{d,t,n}^2}.$$

These stock-level estimates are then averaged to obtain one market-level volatility score for each day:

$$V_d = \frac{1}{N} \sum_{n=1}^{N} RV_{d,n}, \qquad N = 93.$$

The boxplots show substantial overlap between the training, validation, and test distributions. Their medians lie within a narrow range and their interquartile ranges overlap considerably. The validation period shows a slightly higher median than the training and test periods, but the difference is small relative to the within-split variation. The training period contains several high-volatility outliers, which correspond to a broad market sell off in early August 2024. The test period also contains a smaller number of elevated-volatility days, which are linked to the US election in November 2024.

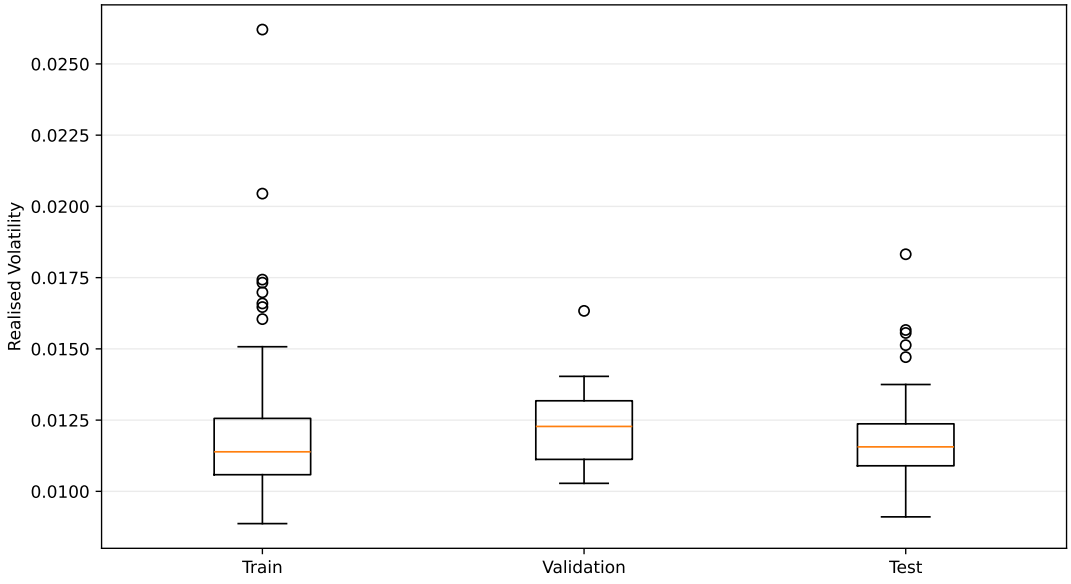


Figure 3.1: Boxplots of average realised volatility per day over all stocks for each data split.

3.1.2 Train Split Analysis

Before introducing the GraphTCN architecture, we first briefly analyse the training data. To examine the dependence structure within the training data, Figure 3.2 presents the Pearson correlation matrix of one-minute Close log returns across the 93 stocks. Stocks are grouped according to their sector classification.

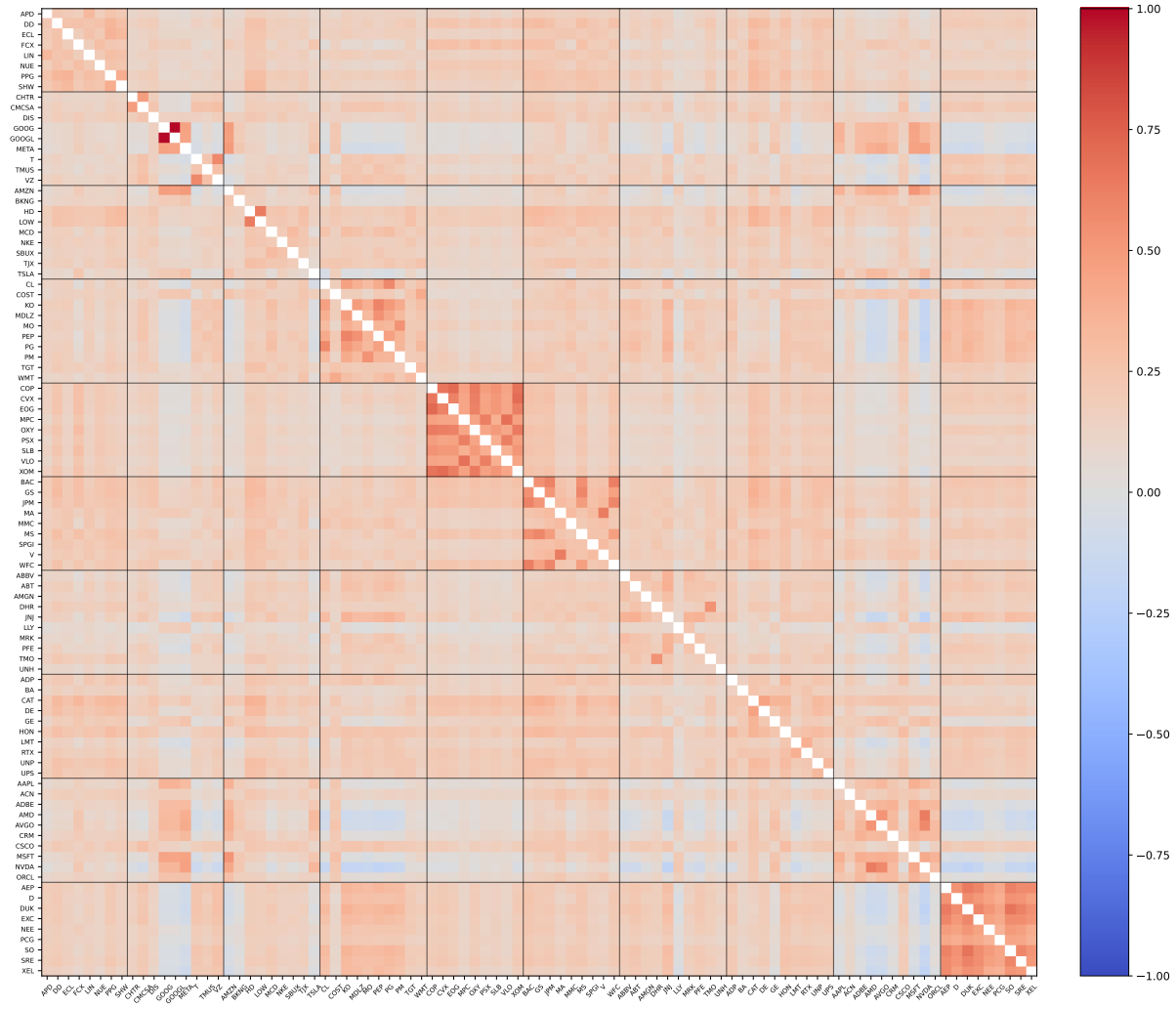


Figure 3.2: Heatmap of Pearson correlation of log returns for all 93 stocks over the entire training split, clustered by sector.

The matrix contains visible blocks of stronger correlation among stocks belonging to the same sector (down the leading diagonal). This is particularly apparent for sectors such as Energy and Utilities, whose constituent companies are exposed to similar economic conditions and industry-specific risk factors. However, the dependence structure is not explained entirely by sector membership. Several pronounced relationships occur between stocks assigned to different sectors. For example, Amazon is classified as Consumer Cyclical but exhibits substantial correlation with several Technology stocks.

The strength of these relationships is not constant across market conditions. Figure 3.3 compares correlation matrices calculated using the lowest and highest 10% of training days according to the daily realised-volatility measure defined previously. The high-volatility heatmap is visibly denser, with stronger positive correlations extending both within and across sectors.

This pattern is consistent with evidence that equity correlations increase during periods of market stress, particularly during broad market sell offs. Ang *et al* find that correlations between US stocks and the aggregate market are substantially greater during downside moves (which usually correspond to periods of higher realised volatility), while Longin *et al* similarly report that correlations increase during bear markets [2, 24]. Intuitively, company-specific information becomes less important when a common market-wide shock dominates. During a sell-off, general risk reduction and deleveraging of portfolios can result in many stocks being sold simultaneously, causing their returns to move together. This provides direct motivation for the dynamic graph formulation of GraphTCN.

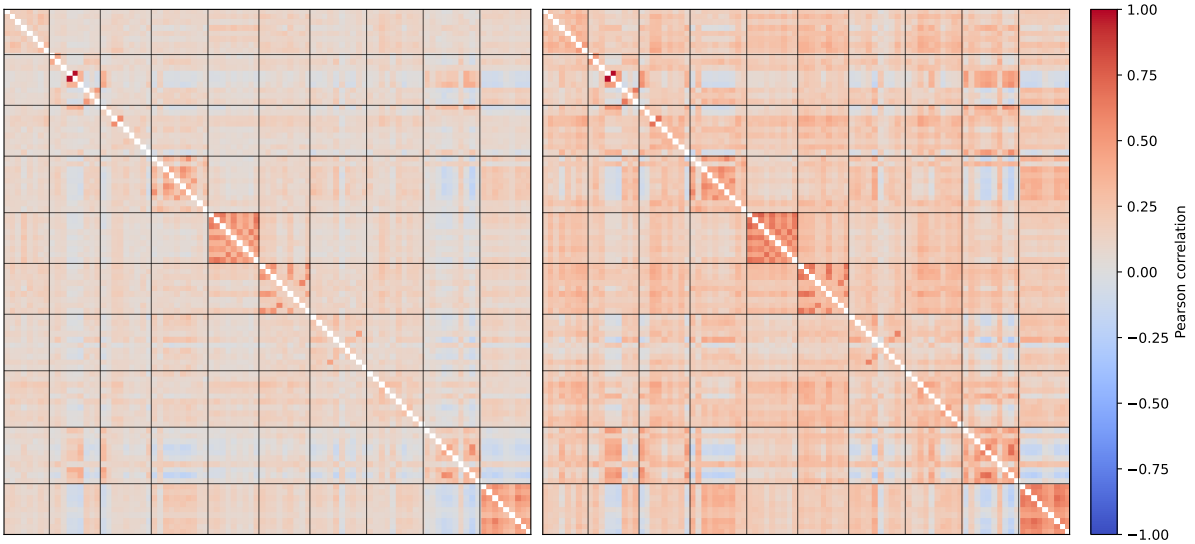


Figure 3.3: Heatmap of Pearson correlation of log returns for the bottom 10% realised volatility days (LHS) and top 10% realised volatility days (RHS).

Having established that cross-stock dependence varies with market conditions, we next investigate the properties of individual return series. Financial returns are commonly characterised by a set of properties known as *stylised facts*. These include a mean close to zero, heavy-tailed return distributions, little autocorrelation in returns, and persistent autocorrelation in squared returns, commonly referred to as volatility clustering [7].

These properties are also present in the training data. The mean one-minute log return across all stocks is close to 0, while the excess kurtosis is large. Since a Gaussian distribution has an excess kurtosis of zero, this large value indicates that extreme returns occur much more frequently than a normal distribution would imply.

Statistic	Value
Mean one-minute log return	0.000093%
Excess kurtosis	30.74

Table 3.4: Summary statistics for one-minute Close log returns in the training split.

Figure 3.5 shows the autocorrelation functions of returns and squared returns. Return autocorrelation falls to close to zero after the first few minutes, demonstrating that return direction contains little persistent linear signal. By contrast, squared returns remain positively autocorrelated over longer lags. Squaring removes the direction of a return while retaining its magnitude, so this slower decay demonstrates volatility clustering: periods of large price movements tend to be followed by further large movements. These characteristics make the financial forecasting problem particularly difficult. Returns are centred close to zero and their direction is only weakly predictable. This makes a simple persistence model a strong baseline.

Another well known artifact of financial data is the intra day volatility curve effect [39], which is the idea that realised volatility is highest in the morning. The reason for this is during the morning trading hours market participants are reacting to overnight news, and overnight trading orders are processed. Realised volatility then dies down during the middle of the day, and picks up again towards the market close as closing auctions take place. The intraday volatility plot for our training data is shown in Figure 3.4. We first compute an intraday rolling 30 minute realised volatility for each stock, which gives an intraday volatility profile per stock per day. These are then averaged to give one profile. We can clearly see the intraday volatility curve effects in our data. We have already seen in our discussion earlier that stocks tend to be more highly correlated in higher volatility regimes, so the intra day effect will give us a useful way to assess our learnt graphs - we should expect them to be denser in the morning when realised volatility is highest.

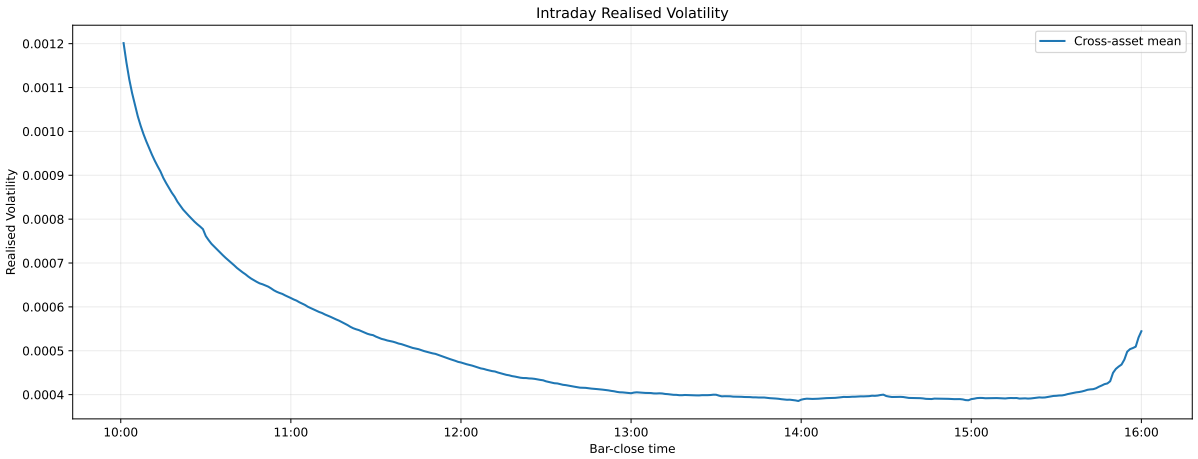
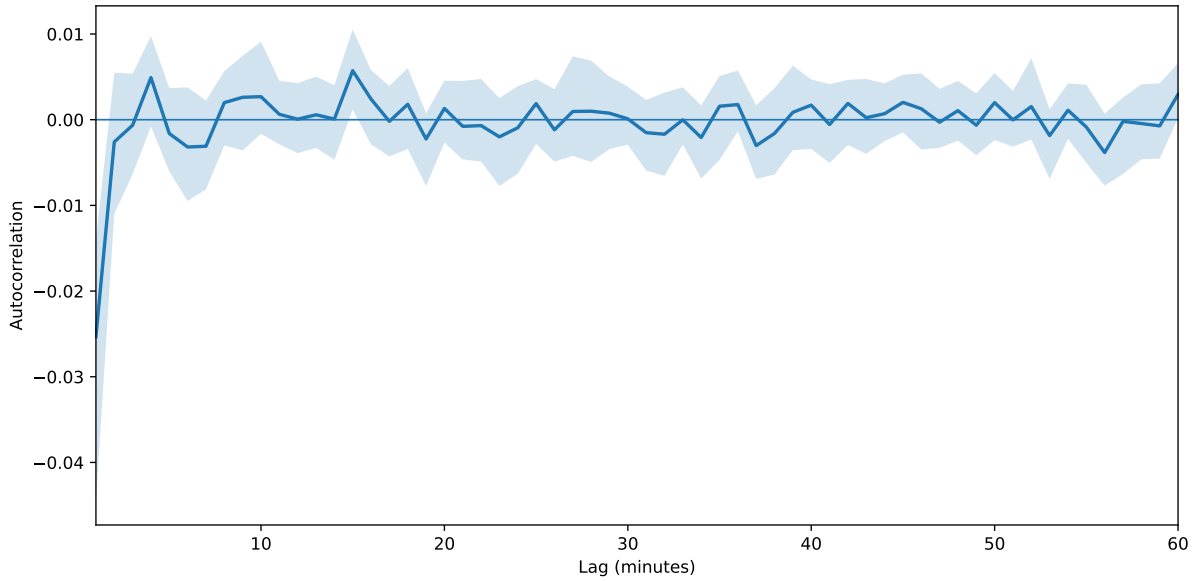
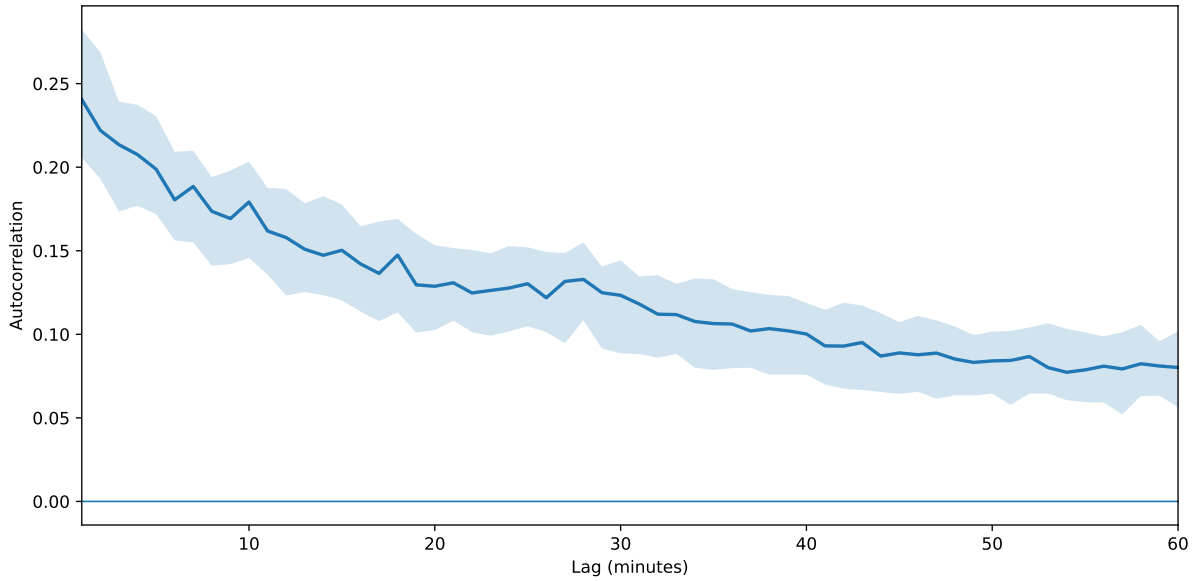


Figure 3.4: Plot of intraday realised volatility averaged over all stocks.



(a) Autocorrelation of one-minute returns.



(b) Autocorrelation of squared one-minute returns.

Figure 3.5: Autocorrelation functions of returns and squared returns in the training split. Returns are calculated within each trading day. The solid line shows the mean across stocks and the shaded region shows the interquartile range.

3.2 GraphTCN

We now introduce the GraphTCN architecture, which is summarised in Figure 3.6. ModernTCN is applied independently to each stock to produce a temporal embedding. In parallel, a direct embedding of the last observed state is constructed. The ModernTCN and direct state embeddings are concatenated and used to learn the dynamic graph. Separately a static graph, initialised with a Close log return correlation matrix, is also learnt. The static and dynamic graphs are combined using a learned gate, giving the final graph. The final graph is then used in the message passing stage, and a final learned gate combines the message passing output with the temporal output from the ModernTCN layer. This final representation is then used to produce direct forecasts at our 5 horizons. We consider each of these steps in more detail.

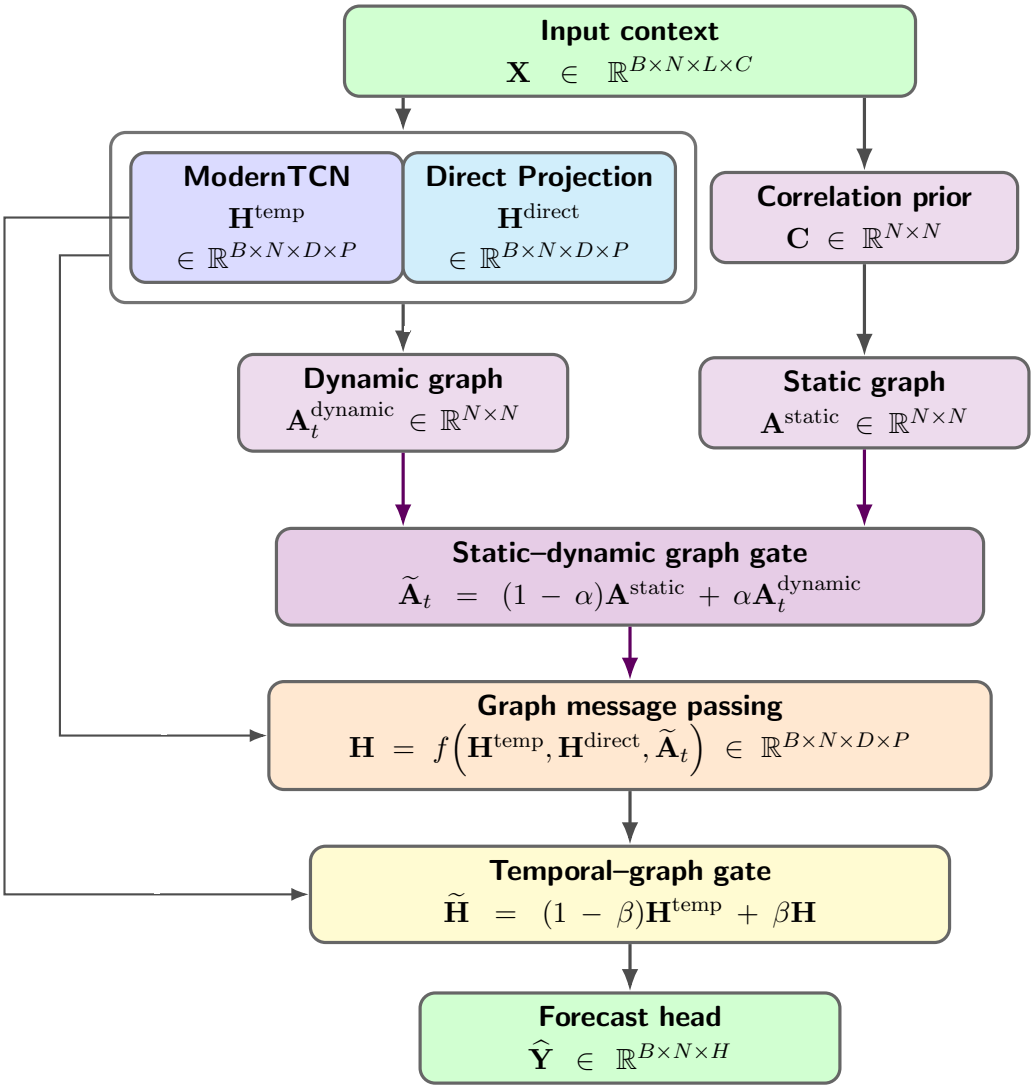


Figure 3.6: GraphTCN architecture.

3.2.1 Temporal and State Representations

For a batch of B forecasting windows, and a context length L the input is written as

$$X \in \mathbb{R}^{B \times N \times L \times C},$$

where $X[b, n, t, :] \in \mathbb{R}^C$ contains the Open, High, Low, Close and Volume observations for stock n at minute t.

The first stage in GraphTCN is to pass our input data through the ModernTCN module and obtain temporal embeddings for each series. We saw in section 2.4.1 that the input data into ModernTCN needs to be of shape $X \in \mathbb{R}^{B \times N \times L}$. To get our data into the correct form, we have 2 options:

- Flatten stock dimension, N, into the input feature dimension, C. This would give $X \in \mathbb{R}^{B \times NC \times L}$
- Flatten stock dimension, N, into the batch dimension, B. This would give $X \in \mathbb{R}^{BN \times C \times L}$

If we choose the first option, the ModernTCN convolutions described in section 2.4.2 would mix information across all input features *and all stocks*. This would mean our ModernTCN module would learn cross asset relationships. In our GraphTCN architecture, cross asset learning is restricted to the spatial module. We therefore flatten the stock dimension directly into the batch dimension, ensuring that input features only mix for the *same* stock, and there is no cross asset learning.

The reshaped data matrix is passed through the ModernTCN module, which is discussed in detail in section 2.4. The final temporal embeddings for each stock are stacked into one matrix and reshaped to give

$$H^{temp} \in \mathbb{R}^{B \times N \times D \times P}.$$

The tensor $H^{temp}[b, n, :, p]$ is therefore the D-dimensional temporal representation of stock n at patch position p.

The ModernTCN module provides one latent representation of each stock. Following the work of Malandreniotis *et al* [27], a second latent representation is constructed in parallel. Both representations are used in dynamic graph construction and message passing.

Let τ_p denote the final minute associated with patch p. The observation at this point is projected into the same D-dimensional space as the ModernTCN temporal representation

$$H^{direct}_{b,n,:,p} = W_S X_{b,n,\tau_p,:} + b_S,$$

where $W_S \in \mathbb{R}^{D \times C}$. This produces

$$H^{direct} \in \mathbb{R}^{B \times N \times D \times P}.$$

The same projection is used for every stock and time point. Unlike H^{temp}, the direct state representation contains no temporal convolution: it is a direct learned embedding of the five variables observed at the end of each patch. H^{temp} and H^{direct} are both used to learn the dynamic graph and for message passing - this allows graph construction and message passing to depend jointly on a compressed representation of the recent trajectory, H^{temp}, and a direct representation of the latest observed state, H^{direct}.

3.2.2 Graph Learning

The next stage in the architecture is to learn the static and dynamic graphs that will be used in the spatial module for message passing. The static graph represents relationships shared across all context windows, while the dynamic graph is generated separately for each context window.

Throughout this section A_{ij} denotes the weight from source stock j to target stock i.

Static graph

The static graph is initialised using the absolute Pearson correlations between one-minute Close returns in the training split. Previous work has shown that a correlation matrix can provide an informative and more stable initialisation for a trainable adjacency matrix than an unstructured random initialisation [21]. Using the return series defined in the section 3.1.1, the correlation prior is

$$C_{ij} = \frac{\left|\mathrm{Corr}\left(r^{(i)}, r^{(j)}\right)\right|}{\sum_{m \neq i} \left|\mathrm{Corr}\left(r^{(i)}, r^{(m)}\right)\right|}, \qquad C_{ii} = 0.$$

We use absolute value since there are stock pairs that have large negative correlations, and intuitively these should be as strongly linked in the graph as stocks that have large positive correlation. The correlation matrix is then mapped to a trainable matrix of initial logits. To do this, we again follow the work of Malandreniotis *et al* [27]. Let

$$\widetilde{C} = \frac{C}{\max_{i,j} C_{ij}}.$$

The initial static logits are constructed as

$$L^{\text{static}} = s\left(\widetilde{C} - \overline{\widetilde{C}}\right) + \epsilon,$$

where

$$s = 4, \qquad \epsilon_{ij} \sim \mathcal{N}(0, 0.02^2)$$

and $\overline{\widetilde{C}}$ denotes the mean of the entries in $\widetilde{C}$. The diagonal is masked and the static adjacency is obtained using a row-wise softmax:

$$A_{ij}^{\text{static}} = \frac{\exp\left(L_{ij}^{\text{static}}\right)}{\sum_{m \neq i} \exp\left(L_{im}^{\text{static}}\right)}, \qquad A_{ii}^{\text{static}} = 0.$$

The parameter s controls the initial separation between weakly and strongly related stocks - a small s compresses the differences between correlations, producing a graph closer to uniform and a large s amplifies those differences, concentrating more weight on the most strongly correlated stocks. Although the original correlation matrix is symmetric, no symmetry constraint is imposed on the learned static graph. The logits L^{static} are optimised during training.

Dynamic graph

The dynamic graph is inferred from the final patch representation of each stock, using both H^{temp} and H^{direct}. To do this, we first concatenate the temporal and direct state representations at the final patch position. Let $\bar{p}$ denote the final patch in a given context window. Then

$$z_{b,n} = H^{temp}_{b,n,:,\bar{p}} \,\|\, H^{direct}_{b,n,:,\bar{p}} \in \mathbb{R}^{2D},$$

where $\|$ denotes concatenation. We then use an attention based graph scorer to learn the dynamic graph from the concatenated representation. Queries and keys are obtained using separate learned projections:

$$q_{b,n} = W_Q z_{b,n} + b_Q, \qquad k_{b,n} = W_K z_{b,n} + b_K,$$

where $W_Q, W_K \in R^{D \times 2D}$. The unnormalised score from source stock j to target stock i is then given by

$$L^{\text{dynamic}}_{b,ij} = \frac{q^\top_{b,i} k_{b,j}}{\sqrt{D}},$$

Self-edges are removed and the scores are normalised across source stocks:

$$A^{\text{dynamic}}_{b,ij} = \frac{\exp\left(L^{\text{dynamic}}_{b,ij}\right)}{\sum_{m \neq i} \exp\left(L^{\text{dynamic}}_{b,im}\right)}, \qquad A^{\text{dynamic}}_{b,ii} = 0,$$

where $A^{dynamic} \in \mathbb{R}^{N \times N}$. Since GraphTCN constructs each dynamic graph using only the final patch in each context window, $\bar{p}$, this means we learn one dynamic graph per context window.

Static–dynamic mixture

Next, before moving on to the message passing stage, we need to combine the static and dynamic graphs. We do this by taking a convex combination of both using a learned parameter. Let $\alpha = \sigma(a)$, where σ is the sigmoid function. The combined graph for a context window is then

$$\tilde{A} = (1 - \alpha) A^{\text{static}} + \alpha A^{\text{dynamic}}.$$

This allows the model to determine how much of the dynamic and static graph to use. A single value of α is shared across all stocks, edges and input windows. Since both

component graphs are non-negative and row-normalised, their convex combination is also non-negative and row-normalised.

3.2.3 Graph Message Passing

The combined graph is then passed to the spatial module for message passing. We first learn a value vector at each patch position, which is a function of both the temporal and direct state representations. For source stock j we have

$$v_{b,j,p} = W_V \left(H^{temp}_{b,j,:,p} \parallel H^{direct}_{b,j,:,p} \right) + b_V, \qquad v_{b,j,p} \in \mathbb{R}^D.$$

$v_{b,j,p}$ is therefore the value of the message that source stock j sends to its neighbours at patch position p. The message received by target stock i is given by

$$m_{b,i,p} = \sum_{j \neq i} \tilde{A}_{b,ij} v_{b,j,p}.$$

The adjacency weight $\tilde{A}_{b,ij}$ determines how strongly source stock j contributes to target stock i, while $v_{b,j,p}$ determines the information that the source transmits. The aggregated message is then projected back into the temporal hidden space and added to the original temporal representation:

$$U_{b,i,p} = \text{LayerNorm} \left(H^{temp}_{b,i,:,p} + W_O m_{b,i,p} \right).$$

The original temporal representation is added here since the diagonal of our combined adjacency matrix is 0, therefore we need this step to ensure we do not lose stock $i's$ own message. A feed-forward network and a second residual connection then produce the graph-updated representation:

$$H_{b,i,:,p} = \text{LayerNorm} \left(U_{b,i,p} + \text{FFN}(U_{b,i,p}) \right).$$

The feed-forward network maps $\mathbb{R}^D \to \mathbb{R}^{2D} \to \mathbb{R}^D$ with a GELU activation, $\text{GELU}(x) = x\Phi(x)$, where Φ is the standard Gaussian cumulative distribution [13].

Next, before passing the graph representation on to the final output layer, we learn

another convex gate which allows us to weight the graph representation and the original temporal representation. The idea here is that we do not want to force our model to use only the graph representation - it may be the case that cross series mixing is actually not useful, and the best forecasts are obtained by only using the temporal representation of each series without cross series learning. This second gate gives the model the flexibility to learn how much of each representation to use. Let $\beta = \sigma(b)$, where σ is again the sigmoid function. The final representation passed to the forecasting head, $\widetilde{H}$, is then given by:

$$\widetilde{H} = (1 - \beta)H^{temp} + \beta H.$$

The two gates therefore have distinct roles: α controls the balance between the static and dynamic graphs, whereas β controls the balance between the pure temporal representation and the graph representation.

3.2.4 Forecasting Head

Finally, the output head, which is shared across all stocks, converts the hidden representation $\widetilde{H}$ into a forecast at our 5 forecast horizons. The complete model output is $\widehat{Y} \in \mathbb{R}^{B \times N \times 5}$, with predictions corresponding to $\mathcal{H} = \{1, 5, 15, 30, 60\}$ minutes. All horizons are predicted directly in parallel, and note that we do not forecast the intermediate time steps.

3.3 Experiments

3.3.1 Forecasting Setup

The same forecasting setup is used for GraphTCN and all baseline models. For each context window w, the model observes the previous 60 one-minute observations for all 93 stocks. Each stock is represented by 5 input features: Open, High, Low, Close and Volume. The input window is therefore

$$X_w \in \mathbb{R}^{N \times L \times C} = \mathbb{R}^{93 \times 60 \times 5}$$

The final observation in X_w defines the forecast origin. From this origin, the model predicts the Close price of every stock at the horizon set $\mathcal{H} = \{1, 5, 15, 30, 60\}$ minutes.

Forecast origins are separated by a stride of 15 minutes. Given 390 observations per trading day and a maximum forecast horizon of 60 minutes, this produces

$$1 + \left\lfloor \frac{390 - L - \max(\mathcal{H})}{15} \right\rfloor = 19$$

forecast windows per complete day. The windows overlap, but every 60-minute context and all of its forecast targets remain within the same trading day. Overnight price changes are therefore excluded, and windows are constructed independently within the training, validation and test splits.

3.3.2 Baselines

GraphTCN is compared with a set of baseline models covering naive, statistical, neural, and pretrained forecasting approaches. They are briefly described and their use motivated.

Persistence

The persistence model predicts that the future Close price of each stock remains equal to its latest observed value:

$$\widehat{C}_{t+h,n} = C_{t,n}, \qquad h \in \{1, 5, 15, 30, 60\}.$$

Equivalently, it predicts a log return of zero at every horizon. The near-zero mean return and weak return autocorrelation observed in the training data suggest that this will be a strong baseline.

Statistical baselines

ARIMA, GARCH, and VAR are included because statistical models provide simple benchmarks against which the additional complexity of neural methods can be assessed and as noted before, more complex machine-learning methods do not consistently outperform

them in financial forecasting tasks [26]. Since these models require stationarity of input data, they are fit to one-minute Close log returns. Their one-step return forecasts are accumulated and converted back to prices using

$$\widehat{C}_{t+h,n} = C_{t,n} \exp\left(\sum_{\tau=1}^{h} \widehat{r}_{t+\tau,n}\right).$$

ARIMA: A separate ARIMA$(p, 0, q)$ model is fit to the return series of each stock, where p and q were optimised by sweeping over a small grid and using the combination that gave the best Akaike Information Criterion (AIC) [1]. Parameters are estimated by maximum likelihood, and since ARIMA predicts autoregressively, predictions are obtained at all time steps out to the maximum horizon of 60 minutes.

GARCH: The persistent autocorrelation in squared returns shown in Figure 3.5 indicates that the variance of returns is not constant. A separate AR(1)–GARCH$(1, 1)$ model is therefore fit to each stock. Note that we did not optimise the order of the GARCH model as the literature suggests that higher order GARCH models do not outperform GARCH$(1, 1)$ in financial time series forecasting [12]. The AR component models the conditional mean return, while the GARCH component models the changing conditional variance using previous squared errors and previous variance estimates. The parameters are estimated jointly by maximum likelihood under Gaussian noise. Like ARIMA, GARCH requires stationarity of the input series, so we again fit it to log return series. It also predicts autoregressively, so predictions are again obtained for all time steps.

VAR: Vector autoregression provides a linear multivariate baseline. A single VAR(p) model is fit jointly to the Close returns of all 93 stocks, allowing the lagged returns of every stock to contribute to every forecast. The lag order, p, is selected by the AIC from a maximum of 15 lags, and the coefficient matrices are estimated by least squares. VAR therefore tests whether fixed linear cross-stock relationships improve forecasting before introducing nonlinear or dynamic graph-based interactions.

ModernTCN

ModernTCN is included as the principal neural baseline and was described in detail in section 2.4. Unlike GraphTCN, the baseline version uses ModernTCN's joint variable

layout. This was the first flattening option described in section 3.2.1. The stock and feature dimensions are flattened as

$$X \in \mathbb{R}^{B\times N\times L\times C} \longrightarrow X^{\text{joint}} \in \mathbb{R}^{B\times NC\times L}.$$

Each stock–feature pair is therefore treated as a separate input variable, allowing ModernTCN's convolutional layers to learn relationships across all stocks and OHLCV features. This provides a non-graph cross-series control and allows us to assess whether learning cross-series through a graph improves on ModernTCN's standard joint-variable formulation.

Kronos

Kronos is included as a pretrained financial time-series foundation-model baseline. The Kronos paper reports state-of-the-art zero-shot performance across several financial forecasting tasks, making it a useful test of whether large-scale pretraining transfers to the present dataset. Its pretraining data cut off is June 2024, meaning it has not been pretrained on our test set so can be used as a baseline [32].

We use the Kronos-small model and its associated tokenizer (`https://huggingface.co/NeoQuasar/Kronos-small`) without fine-tuning. Since Kronos does not learn cross series relationships, each stock is processed separately. Kronos generates forecasts auto regressively, so a prediction is obtained for all time steps. The model learns a distribution over next token, and then ten token-space forecast paths are sampled using temperature 0.6 and nucleus sampling with $p = 0.9$, which are the default settings. The token space forecast paths are then decoded and averaged, and the Close forecasts at 1, 5, 15, 30, and 60 minutes are retained.

Token GraphTCN

Motivated by the token based approach of Kronos, we include a Tokenized GraphTCN version as an additional baseline. We use the frozen pretrained Kronos tokenizer to first convert the continuous price data into the coarse and fine tokens, as described in section 2.5.1. After experimenting, we opted to only retain the coarse token s_1, since we found that adding the fine token did not improve model performance but increased model complexity. The coarse token is first mapped into an embedding space. These embeddings are then processed by the same GraphTCN architecture outlined in section

3.2, except like Kronos, we now learn a distribution over next token.

In order to convert our predicted token sequence back into the original price space, we need to use the frozen Kronos decoder. However since Kronos is an autoregressive model, the decoder expects the full sequence of tokens to decode. We therefore cannot simply predict at our 5 forecast horizons (as we do with GraphTCN) - we need to predict the next token at all steps in between. The tokenized GraphTCN baseline therefore learns the next token distribution at all 60 time steps.

The model is trained using categorical cross-entropy between these distributions and the true future coarse tokens. During inference, we follow the same methodology as Kronos and sample ten complete coarse-token paths, but in experimentation we found using a temperature of 1.0 produces better results. Each path is passed independently through the frozen Kronos decoder to recover a continuous price prediction. The resulting continuous paths are averaged, and the Close forecasts at the five evaluation horizons are retained.

3.3.3 Evaluation Metrics

We now introduce the evaluation metrics used to assess all models on the financial forecasting task. Note that all stocks trade at different price levels, so metrics such as Mean Square Error or Mean Absolute Error in price space would not be appropriate as they would be dominated by stocks that trade at larger price levels. We therefore evaluate the models using metrics that are not impacted by the differing price levels. Each metric is calculated separately at the five forecast horizons.

For forecasting window w, stock n, and horizon h, let $C^{(0)}_{w,n}$ denote the last available Close price in the window, $C_{w,h,n}$ the true future Close price at horizon h , and $\widehat{C}_{w,h,n}$ its prediction. Define the realised and predicted cumulative log returns as

$$r_{w,h,n} = \log\left(\frac{C_{w,h,n}}{C^{(0)}_{w,n}}\right), \qquad \widehat{r}_{w,h,n} = \log\left(\frac{\widehat{C}_{w,h,n}}{C^{(0)}_{w,n}}\right). \tag{3.1}$$

Log MAE: The primary evaluation metric is the mean absolute error between predicted and realised cumulative log returns:

$$\text{LogMAE}_h = \frac{1}{WN}\sum_{w=1}^{W}\sum_{n=1}^{N}\left|\widehat{r}_{w,h,n} - r_{w,h,n}\right|,$$

where W is the number of forecast windows and N is the number of stocks. Expressing the error in log-return space makes forecasts comparable across stocks with different price levels. The absolute error is used because it is less sensitive to extreme observations than squared error. Lower values indicate better forecasting performance.

Win rate: Win rate measures how frequently a model produces a smaller absolute price error than the persistence baseline. The model and persistence errors are

$$e_{w,h,n} = \left|\widehat{C}_{w,h,n} - C_{w,h,n}\right|, \qquad e^{\text{pers}}_{w,h,n} = \left|C^{(0)}_{w,n} - C_{w,h,n}\right|.$$

The win rate is then

$$\text{WinRate}_h = \frac{1}{WN}\sum_{w=1}^{W}\sum_{n=1}^{N}\left[\mathbb{I}\left(e_{w,h,n} < e^{\text{pers}}_{w,h,n}\right) + \frac{1}{2}\mathbb{I}\left(e_{w,h,n} = e^{\text{pers}}_{w,h,n}\right)\right].$$

The win rate for the persistence model is therefore defined to be 0.5. A value above 0.5 indicates that the model has a smaller error than persistence on the majority of forecasting examples. This complements Log MAE by showing whether an improvement is achieved consistently rather than through a small number of unusually accurate forecasts.

Sign accuracy: Sign accuracy measures whether the model correctly predicts the direction of the return:

$$\text{SignAcc}_h = \frac{1}{WN}\sum_{w=1}^{W}\sum_{n=1}^{N}\mathbb{I}\left[\text{sign}\left(\widehat{r}_{w,h,n}\right) = \text{sign}\left(r_{w,h,n}\right)\right].$$

This metric ignores the magnitude of the forecast and considers only whether the predicted movement is positive, negative, or zero. A zero prediction is counted as correct only when the realised return is also exactly zero. Higher values indicate greater directional accuracy.

Pearson correlation: Pearson correlation measures the linear relationship between predicted and realised cumulative log returns. At each horizon, the predictions are pooled across all forecast windows and stocks. Defining

$$\widehat{\mathbf{r}}_h = \text{vec}\left(\left[\widehat{r}_{w,h,n}\right]_{w,n}\right), \qquad \mathbf{r}_h = \text{vec}\left(\left[r_{w,h,n}\right]_{w,n}\right),$$

where the returns are defined in Equation 3.1. The Pearson correlation metric is then calculated as

$$\text{Pearson}_h = \text{Corr}\left(\widehat{\mathbf{r}}_h, \mathbf{r}_h\right).$$

A high value indicates that larger predicted increases and decreases tend to correspond to larger realised movements in the same direction. This metric is undefined for persistence because it predicts a constant price from the end of each forecast window, therefore the cumulative log returns (which recall are computed relative to the last available price in the context window) are all 0 for persistence.

Series Pearson correlation: The previous Pearson metric compares the cumulative return predicted from each forecast origin with the corresponding realised cumulative return. Series-return Pearson instead measures whether changes in the predicted price series follows changes in the realised future-price series.

Let w^- denote the forecast window immediately preceding w within the same trading day. For stock n and horizon h, the changes between consecutive horizon-aligned forecasts are

$$\Delta\widehat{r}_{w,h,n} = \log\left(\frac{\widehat{C}_{w,h,n}}{\widehat{C}_{w^-,h,n}}\right), \qquad \Delta r_{w,h,n} = \log\left(\frac{C_{w,h,n}}{C_{w^-,h,n}}\right).$$

Note the difference between the above definitions and those in Equation 3.1 - the denominators here are the predicted or true values for the same horizon h in the preceding forecast window, whereas in 3.1 the denominator is the last available value in the current context window. Pearson correlation is calculated across forecast windows separately for each stock and then averaged across stocks:

$$\text{SeriesPearson}_h = \frac{1}{N}\sum_{n=1}^{N} \text{Corr}_w\left(\Delta\widehat{r}_{w,h,n}, \Delta r_{w,h,n}\right).$$

A high value indicates that changes in the predicted price series tend to follow changes in the realised series. Note that unlike the previous Pearson correlation metric, this metric is defined for the persistence baseline.

3.3.4 Training and Optimisation

GraphTCN is trained end-to-end using the Adam optimiser [17]. The static graph is initialised from the training-set correlation matrix as described in section 3.2.2. All remaining parameters use the default initialisation of their respective PyTorch or ModernTCN layers. A fixed random seed of 42 is used throughout.

The optimiser separates the parameters into two groups. Parameters that directly construct the adjacency matrix - the query and key projections, the trainable graph logits, and α - use a learning rate of 5×10^{-4}. All remaining parameters, including the ModernTCN temporal module, the direct-state projection, the spatial message-passing module, β, and the forecasting head, use a learning rate of 2.5×10^{-4}. Using different learning rates for different parts of the model was motivated by the idea of *discriminative fine tuning*, which was introduced by Howard *et al* and used in the fine tuning of NLP models [16]. Through experimentation, we found that the larger graph learning rate allows the comparatively small graph-learning module to adapt more rapidly and resulted in better validation performance.

Parameter group	**Learning rate**	**Trainable parameters**
Temporal module parameters	2.5×10^{-4}	39,827
Graph module parameters	5.0×10^{-4}	12,810
Total	–	**52,637**

Table 3.5: Trainable parameters in GraphTCN, divided according to their optimiser parameter group.

We also employ a LR schedule in training; both learning rates remain fixed for the first 15 epochs and are then multiplied by 0.9 after each subsequent epoch. Models are trained using a batch size of 16 for at most 100 epochs. The training objective is a scaled version of the Log MAE discussed in section 3.3.3. This is used instead of a price space MAE or MSE to ensure the loss is not dominated by stocks that have larger price scales. The same loss is used to train the ModernTCN baseline, ensuring that differences between the two models are not caused by different optimisation objectives.

After every epoch, performance is evaluated on the validation split. Training stops when this validation loss has not improved for 10 consecutive epochs, and the checkpoint with the lowest validation score is retained for final evaluation. All training was done on Google Colab using an Nvidia T4 GPU with 16GB of VRAM.

3.3.5 Hyperparameter Selection

Hyperparameters were selected through a staged search, with one group of parameters fixed before the next was varied. This was due to the fact that compute was limited and a full sweep over all possible combinations would have been prohibitively expensive. All hyperparameters were chosen based on validation set performance.

For the ModernTCN components of GraphTCN, we considered hidden dimensions $D \in \{32, 64, 128\}$, one or three ModernTCN blocks, patch-size and stride pairs of $(4, 2)$ and $(8, 4)$, and large kernels $K^{\text{large}} \in \{15, 51\}$. The small kernel was fixed at 5. Learning rates of 5×10^{-5}, 1×10^{-4}, and 2.5×10^{-4} were evaluated.

We also compared four graph formulations: a trainable static graph, a trainable static-dynamic mixture with a randomly initialised static graph, a static-dynamic mixture whose static graph was initialised from the training-set correlation matrix, and a fixed correlation graph. Graph learning rates of 5×10^{-4}, 1×10^{-3}, and 2×10^{-3} were evaluated separately from the temporal module learning rate. The wider architecture search also considered between one and four spatio-temporal blocks, where a spatio-temporal block is simply defined as a full temporal, graph learning and spatial message passing block. Each stacked spatio-temporal block therefore learns its own graph.

The selected GraphTCN used $D = 32$, one spatio-temporal block, a patch size of 8, a stride of 4, $K^{\text{large}} = 15$, and a correlation-initialised static–dynamic graph. The final architectural settings for GraphTCN are summarised in Table 3.6

Component	**Setting**
Context length L	60 minutes
Number of stocks N	93
Input features C	5
Hidden dimension D	32
Patch length	8
Patch stride	4
Number of patch positions P	15
Large and small kernels	15 and 5
Forecast horizons $\mathcal{H}$	$\{1, 5, 15, 30, 60\}$ minutes

Table 3.6: Architectural settings of GraphTCN.

Chapter 4

Results and Discussion

This chapter presents and discusses the empirical results of the study. We first evaluate GraphTCN and the baseline models on the financial dataset, examining forecasting accuracy, the behaviour of the learned graphs, and the contribution of the main architectural components through ablation experiments. We then evaluate GraphTCN on the weather benchmark to determine whether the architecture can exploit clearer temporal and spatial relationships in a less noisy forecasting setting. The chapter concludes by summarising the principal findings across both datasets.

4.1 Finance Dataset

4.1.1 Forecasting Results

Table 4.1 presents the forecasting results on the financial test set. The primary Log MAE metric is shown in basis points (bps), which is computed by multiplying the original metric by 10,000. The most striking result is the strength of persistence - no model achieves a lower Log MAE at any of the five forecast horizons. This result is consistent with the properties of the data identified in Section 3.1.2. One-minute returns are centred close to zero and contain little persistent autocorrelation, so the latest observed price is already a strong estimate of the future price. This conclusion is also reinforced by the Win Rate results - every evaluated model has a Win Rate below 50%, meaning that no model beats persistence more frequently than it loses at the individual forecast level.

To summarise how the models perform across all horizons, Table 4.2 shows the average Log MAE across the 5 horizons.

Horizon	Model	Log MAE (bps) ↓	Win Rate ↑	Sign Acc. ↑	Pearson ↑	Series Pearson ↑
1	Persistence	**3.666**	**50.00%**	10.66%	–	**0.91878**
	ARIMA	3.673	45.84%	45.15%	0.03171	0.91864
	VAR	3.790	43.45%	45.00%	0.01087	0.91584
	GARCH	3.675	45.69%	45.29%	0.03194	0.91861
	ModernTCN	3.744	44.22%	45.07%	0.01383	0.91726
	Kronos	4.090	42.31%	**45.70%**	0.00216	0.88562
	Token GraphTCN	4.008	41.59%	45.32%	0.00699	0.90499
	GraphTCN	3.683	45.56%	45.58%	**0.04959**	0.91877
5	Persistence	**7.850**	**50.00%**	3.31%	–	**0.64525**
	ARIMA	7.854	48.63%	48.20%	0.01596	0.64410
	VAR	7.991	46.98%	**48.99%**	0.01578	0.63855
	GARCH	7.854	48.56%	48.37%	0.01438	0.64423
	ModernTCN	7.956	47.25%	48.76%	0.01049	0.63876
	Kronos	8.239	46.00%	48.93%	0.01056	0.60116
	Token GraphTCN	8.055	46.34%	48.69%	0.01132	0.62539
	GraphTCN	7.856	48.77%	48.86%	**0.02193**	0.64336
15	Persistence	**13.220**	**50.00%**	1.92%	–	-0.00330
	ARIMA	13.229	48.96%	48.76%	-0.00129	-0.00336
	VAR	13.354	47.59%	49.04%	-0.00254	-0.00340
	GARCH	13.232	48.75%	48.79%	0.00094	-0.00335
	ModernTCN	13.270	48.33%	49.19%	0.01422	**-0.00136**
	Kronos	13.715	46.06%	48.84%	-0.01582	-0.00319
	Token GraphTCN	13.360	47.47%	49.00%	-0.00009	-0.00215
	GraphTCN	13.224	49.26%	**49.43%**	**0.03717**	-0.00161
30	Persistence	**18.384**	**50.00%**	1.38%	–	-0.01250
	ARIMA	18.400	48.97%	48.99%	0.00069	-0.01239
	VAR	18.477	48.28%	49.41%	0.00438	-0.01217
	GARCH	18.407	48.77%	49.07%	0.00278	-0.01242
	ModernTCN	18.451	48.57%	**49.62%**	0.01333	**-0.01071**
	Kronos	18.975	46.07%	48.93%	-0.01726	-0.01224
	Token GraphTCN	18.516	47.85%	49.17%	-0.00099	-0.01330
	GraphTCN	18.397	49.28%	49.50%	**0.01657**	-0.01199
60	Persistence	**25.534**	**50.00%**	0.94%	–	0.00125
	ARIMA	25.580	48.51%	48.85%	-0.00053	0.00123
	VAR	25.627	48.50%	49.46%	0.00226	0.00195
	GARCH	25.602	48.21%	48.90%	0.00341	0.00118
	ModernTCN	25.661	48.22%	49.51%	0.00657	**0.00268**
	Kronos	26.449	45.99%	49.30%	-0.02677	-0.00048
	Token GraphTCN	25.701	47.77%	49.03%	-0.01277	0.00005
	GraphTCN	25.555	49.24%	**49.74%**	**0.00759**	0.00156

Table 4.1: Forecasting results on the financial test set. Log MAE is reported in basis points. Win Rate and Sign Accuracy are reported as percentages, while both Pearson metrics are reported as correlation coefficients. The best result at each horizon is shown in bold and the second-best is highlighted.

Model	Mean Log MAE (bps)
Persistence	13.731
GraphTCN	13.743
ARIMA	13.747
GARCH	13.754
ModernTCN	13.816
VAR	13.848
Token GraphTCN	13.928
Kronos	14.294

Table 4.2: Unweighted mean Log MAE across the five forecast horizons.

We can see that although GraphTCN does not outperform persistence overall, it has the lowest average Log MAE of the remaining models across the 5 horizons. The difference versus ARIMA and GARCH is small, but the pattern across horizons is informative. At 1 and 5 minutes, the univariate statistical models remain marginally more accurate. At 15, 30, and 60 minutes, GraphTCN achieves the lowest Log MAE of every non-persistence model.

ARIMA and GARCH remain close to persistence because their conditional-mean forecasts are generally small. This is advantageous at the shortest horizons, where price changes are close to zero. Their relative advantage decreases as the horizon increases, while GraphTCN has more opportunity to use temporal and cross-stock information accumulated over the context window. VAR performs less strongly, suggesting that unrestricted fixed linear relationships across all 93 stocks are not sufficient to improve the forecasts.

Persistence's dominance over the full test set raises a question: is its advantage concentrated among lower volatility stocks whose prices move very little, leaving limited scope for a learned model to improve upon a no-change forecast? To investigate this, we compare performance of the models over the most and least volatile stocks. For every stock, we compute its realised volatility over the test period and then rank the stocks from most to least volatile. We then take the top and bottom quartile of stocks and compute the Log MAE of each model minus the Log MAE of persistence on each stock subset. A negative value therefore means that the model has a lower Log MAE than persistence. Table 4.3 reports the findings.

The results support the hypothesis that persistence is particularly difficult to beat when volatility is low. Every model performs worse than persistence for the least volatile stocks at every horizon. However, the pattern changes for the most volatile stocks. GraphTCN is the strongest model at every horizon beyond one minute and consistently improves upon persistence from 5 to 60 minutes, with the improvement getting larger as the forecast horizon gets longer. GraphTCN is the only model to beat persistence at every forecast horizon beyond 1 minute. The result provides evidence that the cross-stock learning using graphs introduced by GraphTCN becomes most useful when the volatility of the underlying series is high and therefore the persistence prediction is less effective.

Table 4.1 also shows GraphTCN achieves the highest Pearson correlation at every forecast horizon. The correlations are small, but this is expected in a forecasting problem where returns contain little persistent signal. We note however that GraphTCN consistently captures more of the variation in future returns than the alternative models.

Model	1 min	5 min	15 min	30 min	60 min
Least volatile stocks					
ARIMA	0.005	0.007	0.017	0.038	0.102
VAR	0.065	0.078	0.085	0.086	0.133
GARCH	0.006	0.006	0.014	0.031	0.092
ModernTCN	0.051	0.071	0.026	0.046	0.118
Kronos	0.300	0.256	0.320	0.403	0.684
Token GraphTCN	0.243	0.159	0.106	0.109	0.131
GraphTCN	0.016	0.012	0.006	0.002	0.015
Most volatile stocks					
ARIMA	0.004	**-0.002**	0.000	**-0.001**	0.000
VAR	0.166	0.274	0.297	0.099	0.123
GARCH	0.009	0.001	0.008	0.027	0.064
ModernTCN	0.126	0.201	0.047	0.047	0.131
Kronos	0.642	0.640	0.823	1.011	1.582
Token GraphTCN	0.577	0.339	0.177	0.188	0.186
GraphTCN	0.010	**-0.003**	**-0.014**	**-0.015**	**-0.022**

Table 4.3: Log MAE relative to persistence for the least and most volatile stocks. Values are reported in basis points. Negative values, shown in bold, indicate lower error than persistence.

Since the correlations are all fairly close to 0, we assess the uncertainty in these estimates by calculating 95% confidence intervals using 10,000 block-bootstrap samples of the test set. Complete trading days were sampled with replacement. The resulting intervals are shown in Table 4.4, with intervals that exclude zero in bold.

	1 min		5 min		15 min		30 min		60 min	
Model	**Lower**	**Upper**	**Lower**	**Upper**	**Lower**	**Upper**	**Lower**	**Upper**	**Lower**	**Upper**
ARIMA	**0.003**	**0.056**	0.000	0.032	−0.015	0.012	−0.014	0.015	−0.022	0.020
VAR	−0.005	0.027	0.000	0.031	−0.018	0.013	−0.012	0.021	−0.013	0.017
GARCH	**0.006**	**0.055**	−0.002	0.031	−0.015	0.016	−0.010	0.015	−0.010	0.018
ModernTCN	−0.008	0.034	−0.007	0.028	−0.006	0.034	−0.010	0.038	−0.025	0.036
GraphTCN	**0.029**	**0.071**	**0.008**	**0.036**	**0.013**	**0.061**	−0.006	0.039	−0.034	0.044
Kronos	−0.047	0.039	−0.010	0.031	−0.034	0.002	−0.034	0.000	−0.066	0.008
Token GraphTCN	−0.030	0.037	−0.004	0.027	−0.026	0.024	−0.021	0.019	−0.042	0.015

Table 4.4: The 95% block-bootstrap confidence intervals for the Pearson correlation between predicted and realised cumulative log returns. Intervals are calculated using 10,000 bootstrap samples of complete trading days. Bounds are reported to three decimal places. Intervals that exclude zero, determined using the unrounded values, are shown in bold.

We see that most of the intervals contain zero. GraphTCN provides the most consistent evidence of a positive relationship between predicted and realised returns. Its confidence interval lies entirely above zero at the 1-, 5-, and 15-minute horizons. The result at 5 and 15 minutes are particularly notable because GraphTCN is the only model whose

interval remains entirely positive. At 30 and 60 minutes, however, every confidence interval contains zero. The positive GraphTCN point estimates at these horizons should therefore be treated as uncertain rather than as clear evidence of return predictability.

Another striking feature of Table 4.1 is the poor performance of the two token-based models. Kronos is the worst performing model, while Token GraphTCN is the second-worst model overall. Token GraphTCN improves upon Kronos at every horizon, indicating that cross-stock learning provides some value in token space, but it remains clearly behind the continuous GraphTCN architecture.

To understand this result, Table 4.5 evaluates the token predictions produced by Token GraphTCN before they are decoded back into continuous prices. Top-k accuracy measures the proportion of forecasts for which the true token is among the k tokens assigned the highest probability by the model. The *Excess* column reports the improvement, in percentage points, over a frequency-based baseline that always selects the k most common tokens in the training set. For example, a Top-10 excess of 21.07 percentage points at the 1 minute horizon means that the true token appears among the model's ten highest-probability predictions 21.07 percentage points more often than under the frequency-based baseline.

Horizon	**Top-1**		**Top-5**		**Top-10**	
(min)	**Accuracy**	**Excess**	**Accuracy**	**Excess**	**Accuracy**	**Excess**
1	11.35	6.69	33.78	16.80	48.78	21.07
5	7.91	3.50	26.04	10.04	39.60	13.35
15	6.32	2.60	21.24	7.37	33.05	9.82
30	5.81	2.56	19.87	6.53	31.01	9.27
60	5.89	2.94	19.97	6.17	30.94	7.96

Table 4.5: Token prediction accuracy for Token GraphTCN. Accuracy is reported as a percentage and Excess values are the improvement of the model versus predicting the most common k tokens from the training set.

Recall that there 1,024 different possible tokens, so the results in Table 4.5 show that the model has genuine predictive power in token space. At one minute, the correct token is ranked first in 11.35% of cases and appears among the ten most probable tokens in 48.78% of cases. Predictability declines with the forecasting horizon, but every reported Top-k accuracy has a positive Excess.

Since the token prediction performance is strong, we hypothesise that the cause of the poor performance of the token models therefore comes from the decoding step. The training of the Kronos tokenizer involves encoding price data and then immediately decoding it. This means the decoder is trained to decode a perfect sequence of tokens. During forecasting, however, it receives imperfect token sequences generated by the forecasting

model. To measure its sensitivity to these errors, we carry out a controlled experiment where we first encode the price data and then randomly corrupt a fixed percentage of the tokens. We then decode the corrupted sequence and measure the reconstruction loss as well as the correlation of the decoded series with the original series. The results are shown in Table 4.6.

Corrupted tokens (%)	**Log MAE (bps)**	**Correlation**
0	1.661	0.879
10	3.974	0.464
20	5.854	0.337
30	7.373	0.263
40	8.508	0.212
50	9.350	0.168

Table 4.6: Sensitivity of the frozen Kronos decoder to corruption of the token sequence. The uncorrupted reconstruction is shown in the first row.

The uncorrupted token sequence is reconstructed extremely accurately, however we observe that the decoder is highly sensitive to token errors. Corrupting only 10% of the sequence more than doubles the Log MAE reconstruction error and almost halves the correlation with the original series. As the corruption rate increases, reconstruction quality deteriorates rapidly. This explains why the performance of both token based models is so poor, and suggests that post training the decoder so that it learns to decode the imperfect sequence of tokens it receives from the forecasting model may help improve performance. We leave this as a direction for future work.

Overall, the financial forecasting results provide an answer to the first two research questions from Section 1.2. GraphTCN consistently improves upon ModernTCN and is the strongest non-persistence model, with its clearest advantage occurring among high-volatility stocks. However, it does not outperform persistence across the full stock universe.

4.1.2 Graph Behaviour

We now examine the learned graphs directly to determine whether they contain interpretable cross-stock structure and whether their behaviour changes with market conditions.

We first examine the learned α and β gate parameter values described in Section 3.2.2. Table 4.7 reports the learned values of the two graph gates from a random initialisation.

Recall α controls the balance between the static and dynamic adjacency matrices, while β controls the balance between the original temporal representation and the graph-updated representation.

Parameter	Learned value
α	0.5105
β	0.4767

Table 4.7: Learned values of the GraphTCN graph gates.

Both parameters settle close to 0.5. The learned value of α assigns approximately equal weight to the static and dynamic graphs, while the learned value of β retains substantial contributions from both the temporal and graph-updated representations. Crucially, neither pathway is suppressed during training, therefore the model makes use of both graphs and retains the temporal representation learnt by the ModernTCN temporal module.

Figure 4.1 compares the static, dynamic and combined graphs learnt over the test set. The static graph is shared across all forecast windows. Since we learn a different dynamic graph per context window, we report the mean of the individual dynamic graphs, while the final graph panel shows the mean mixed graph that was used for message passing. The averages are calculated over all 1,178 test windows.

The learned static graph retains clear sector-based structure. Strong blocks are visible within Energy, Consumer Defensive, Technology and Utilities sector, together with several economically plausible cross-sector relationships. This is expected to some extent because the static graph was initialised using absolute return correlations. However, its parameters remained trainable and were updated during training.

Table 4.8 quantifies the similarity between each learned graph component and the original training-set correlation prior by computing the correlation between the final learnt graph components and the prior.

Graph component	Pearson	Spearman
Final mixed graph	0.772	0.658
Static graph	0.778	0.811
Mean dynamic graph	0.334	0.193

Table 4.8: Similarity between the learned graph components and the training-set absolute-correlation prior.

We observe the static graph retains much of the information from the correlation prior, as shown by its Spearman correlation of 0.811, but it is not identical. The mean dynamic graph is much less closely related to the correlation prior, indicating that it captures a

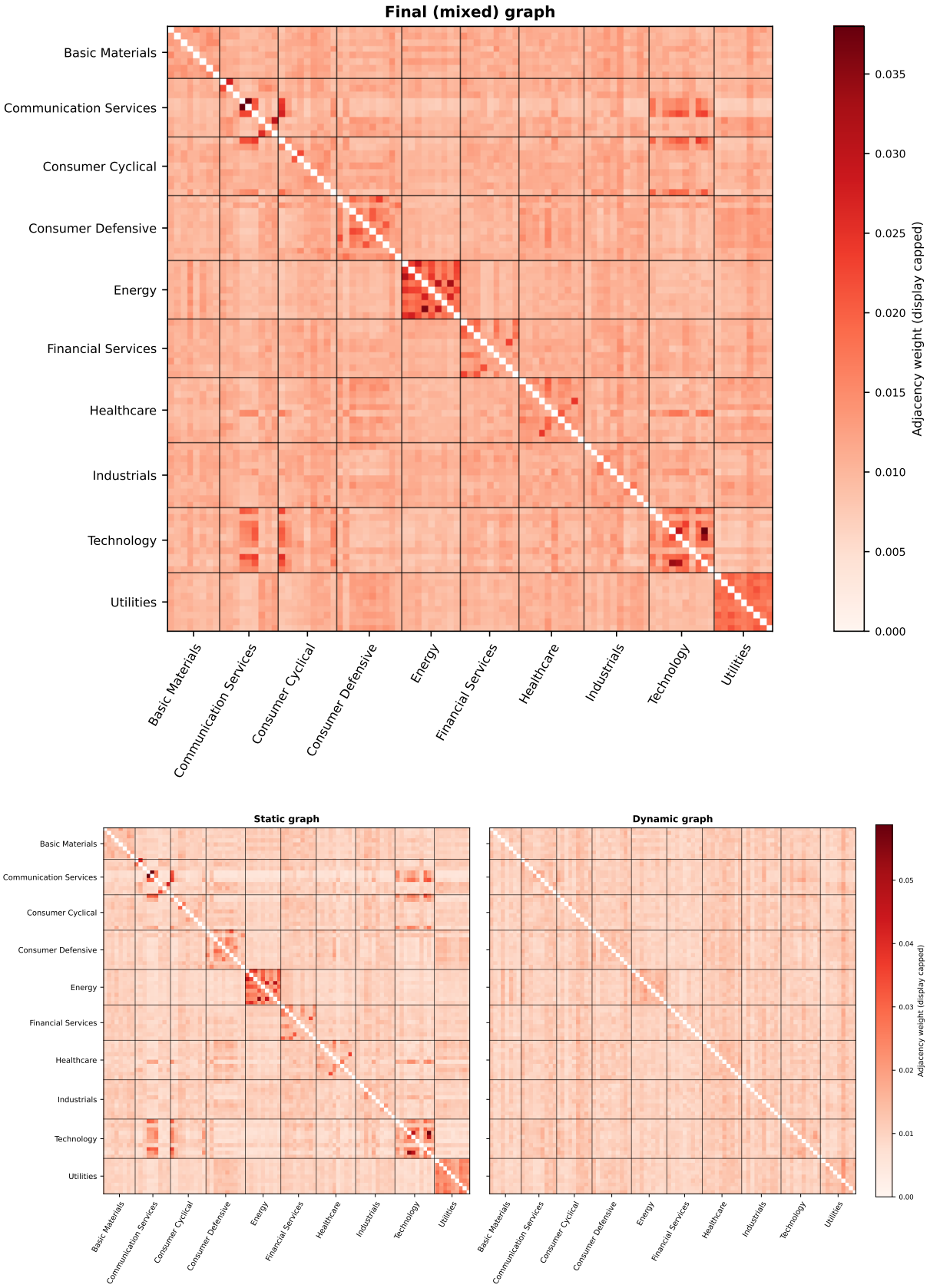


Figure 4.1: Average learned graph components across all test windows. The top panel shows the final mixed graph used for message passing. The lower panels show the learned static graph and the mean context-dependent dynamic graph. Stocks are grouped by sector, rows represent target stocks, columns represent source stocks, and self-edges are excluded.

different set of relationships. The final mixed graph preserves the broad correlation structure while allowing the dynamic component to alter the relative importance of individual edges.

We note in Figure 4.1 that the average dynamic graph appears relatively diffuse - we don't see the same strong structure that we see in the static graph. However, this does not mean that the dynamic graph is diffuse within an individual forecast window. Averaging combines graphs in which different source stocks become important at different times, causing their weights to be spread across the mean adjacency.

Figure 4.2 shows an example of an individual dynamic graph from the training period. We see that graph weight is concentrated on a comparatively small set of stocks. This implies that the dynamic graph is learning specific stock relationships throughout the trading day, and the nature of these relationships changes with each context window and each day.

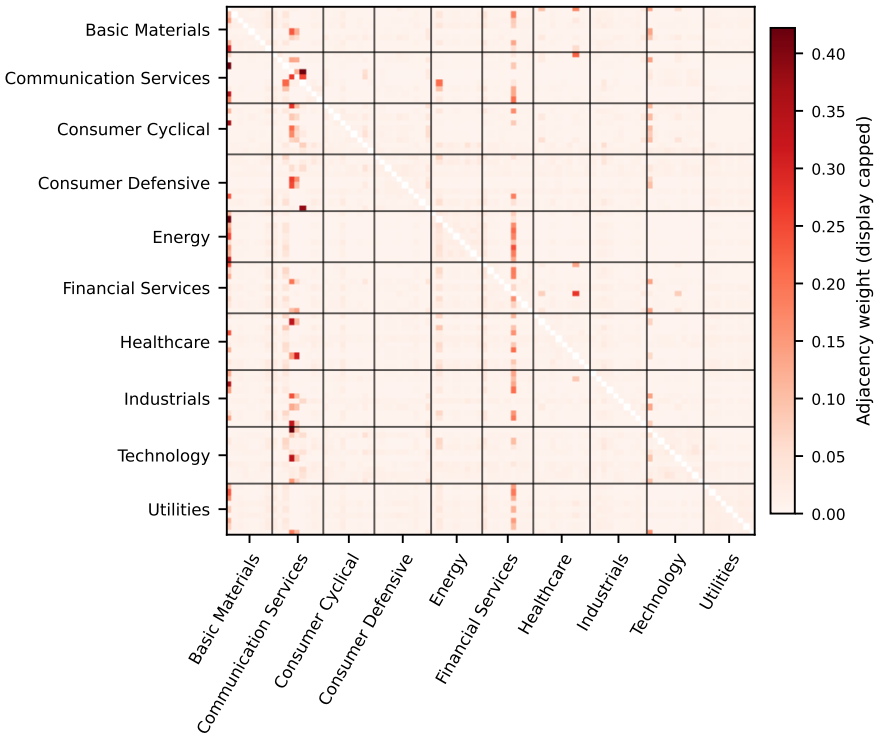


Figure 4.2: Example of a learnt graph on 18th March 2024 at 11:45am

We can demonstrate the economic plausibility of these relationships with two illustrative examples. On 18 March 2024, reports stated that Apple was considering licensing Alphabet's Gemini model for future iPhone functionality [8]. The dynamic edge from Alphabet's GOOG stock to Apple ordinarily received little weight, with a training-period average of 0.0017. At the 11:45 on the announcement day, its dynamic weight increased to 0.797, making GOOG Apple's most influential stock in that window. After combina-

tion with the static graph, the edge retained a mixed weight of 0.422. A second example occurs on 26 March 2024, when Visa and Mastercard announced a joint settlement concerning merchant interchange fees [35]. The dynamic weight from Visa to Mastercard increased from a training-period average of 0.0027 to 0.685. The corresponding weight in the final mixed graph was 0.367.

Another way of measuring the changing nature of the graphs is to compute graph entropy. For a given forecast window w, the entropy of the graph is calculated by first finding the entropy of each row and then averaging across over rows:

$$\mathcal{E}_w = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq i} A_{w,ij}\log A_{w,ij}.$$

Low entropy indicates that a stock assigns most of its weight to a small number of other stocks, while high entropy indicates that its weight is distributed more evenly. With 92 possible source stocks, the maximum row entropy is given by $\log(92) = 4.522$. The entropy of the mean dynamic graph is 4.510, which is close to this maximum. In contrast, the average entropy calculated separately for the individual training-window graph is 2.598. Individual dynamic graphs are therefore substantially more concentrated than their average suggests.

The previous analysis shows that our learned graphs change through time. One question of interest is to understand how the graphs react to the changing intraday realised volatility shown in Figure 3.4. As discussed before, stock returns tend to become more correlated in higher volatility regimes, and we should therefore expect our learned graphs to reflect this. Figure 4.3 shows the average entropy of the graphs as a function of the time of day. Entropy is first calculated for each individual graph and then averaged across graphs sharing the same forecast-origin time. We observe that graph entropy is highest in the morning, meaning that the learned graphs are denser when realised volatility is high. We also note the intraday entropy profile looks very similar to the intraday realised volatility profile in Figure 3.4, indicating that the density of the graphs are reacting to realised volatility.

Taken together, the learned gates, graph structure and entropy profile provide evidence that GraphTCN learns both persistent and adaptive cross-stock relationships. The static graph retains the broad structure of the correlation prior, while the dynamic graph changes its concentration and edge assignments across forecast windows.

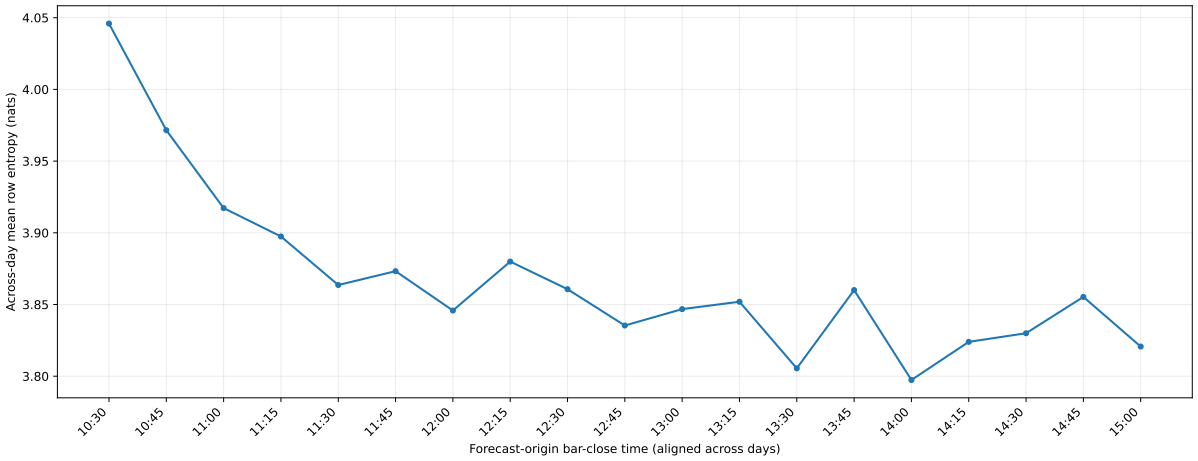


Figure 4.3: Intraday entropy profile of learned graphs.

4.1.3 Ablation Study

The previous section showed that GraphTCN learns both persistent and context-dependent relationships between stocks. We now compare alternative graph formulations to assess whether both graph components of the final architecture are useful. All variants use the same selected ModernTCN temporal configuration and forecasting setup, while changing how the graph used for message passing is constructed.

The *static graph* variant learns a single adjacency matrix that is shared across all forecast windows. The *dynamic-only graph* instead generates a separate adjacency matrix from each input window, without retaining a static component. The *static + dynamic (no prior)* variant combines these two components, but initialises the static graph without the correlation prior - we instead intialise randomly. The *correlation-only graph* uses a fixed graph constructed from the absolute training-return correlations, which is not optimised during training. Variants containing both static and dynamic graph components use the learned gate α, whereas the single-component variants do not require a mixing gate.

Table 4.9 shows that the GraphTCN architecture achieves the lowest Log MAE at every forecast horizon, making it the most consistently accurate graph formulation. Neither graph component is as effective when used alone. The static graph is generally stronger than the dynamic-only graph, particularly beyond the one-minute horizon, supporting the argument that estimating the complete graph independently from each short and noisy context window can be unstable. However, the correlation-only graph also remains worse than GraphTCN, showing that persistent correlation structure alone is insufficient.

The comparison with the static-dynamic model without a prior is particularly informative. Adding a dynamic graph to an unstructured static graph does not match the performance of the complete model, whereas initialising the static component from the training-return

Horizon	Model	Log MAE (bps) ↓	Win Rate ↑	Sign Acc. ↑	Pearson ↑	Series Pearson ↑
1	GraphTCN	**3.683**	**45.56%**	**45.58%**	**0.04958**	**0.91877**
	Static graph	3.696	45.11%	45.43%	0.01293	0.91781
	Dynamic-only graph	3.695	45.17%	45.51%	0.02241	0.91793
	Static + dynamic (no prior)	3.698	45.21%	45.55%	0.01307	0.91807
	Correlation-only graph	3.694	45.10%	45.43%	0.01778	0.91794
5	GraphTCN	**7.856**	**48.77%**	**48.86%**	**0.02192**	**0.64336**
	Static graph	7.869	48.27%	48.62%	0.00620	0.64244
	Dynamic-only graph	7.871	48.27%	48.64%	0.00241	0.64232
	Static + dynamic (no prior)	7.866	48.41%	48.80%	0.01228	0.64281
	Correlation-only graph	7.865	48.51%	48.85%	0.01074	0.64269
15	GraphTCN	**13.224**	49.26%	49.43%	**0.03717**	-0.00161
	Static graph	13.225	**49.26%**	**49.60%**	0.03340	**-0.00139**
	Dynamic-only graph	13.234	48.91%	49.25%	0.02131	-0.00190
	Static + dynamic (no prior)	13.231	48.95%	49.27%	0.01952	-0.00145
	Correlation-only graph	13.231	49.03%	49.34%	0.01871	-0.00210
30	GraphTCN	**18.397**	**49.28%**	**49.50%**	0.01657	-0.01199
	Static graph	18.407	48.85%	49.33%	**0.01736**	**-0.01162**
	Dynamic-only graph	18.414	48.69%	49.20%	0.00674	-0.01262
	Static + dynamic (no prior)	18.415	48.75%	49.22%	0.00655	-0.01193
	Correlation-only graph	18.410	48.93%	49.34%	0.00059	-0.01274
60	GraphTCN	**25.555**	**49.24%**	**49.74%**	0.00759	**0.00156**
	Static graph	25.574	48.57%	49.16%	**0.01121**	-0.00063
	Dynamic-only graph	25.583	48.68%	49.32%	0.00401	0.00002
	Static + dynamic (no prior)	25.584	48.60%	49.19%	-0.00868	0.00036
	Correlation-only graph	25.570	48.91%	49.47%	-0.00002	-0.00003

Table 4.9: Forecasting results for alternative GraphTCN graph formulations. The best result at each horizon is shown in bold and the second-best is highlighted. Rankings are determined using the unrounded values.

correlations gives lower Log MAE at every horizon. The results therefore support the combined design: the correlation prior supplies a stable starting structure, while the trainable static and context-dependent dynamic components allow that structure to be refined. Overall, the point estimates support the use of a correlation initialised static-dynamic graph over fixed, static-only, dynamic-only, or unstructured static-dynamic alternatives.

4.2 Weather Dataset

The financial experiments showed that GraphTCN improves upon the other learned models, but also demonstrated how difficult it is to outperform persistence when returns contain little predictable structure. This leaves an important question: does the limited improvement reflect a weakness in the graph architecture, or is it simply a function of the low signal-to-noise ratio of financial data? To distinguish between these explanations, GraphTCN is evaluated on a spatial weather benchmark. This benchmark was introduced by Shu *et al* and was used to evaluate Sonnet, a state of the art time series forecasting model [33]. Weather measurements contain clearer temporal patterns and neighbouring locations are connected through physical atmospheric processes, providing a more structured setting in which to test whether the model can exploit cross-series relationships.

The benchmark covers the period from 1979 to 2018 and consists of 5 datasets, each containing six-hourly observations for Cape Town, Hong Kong, London, New York and Singapore. Each dataset contains five variables recorded at nine locations arranged in a 3×3 spatial grid. The forecasting task is to predict one of the variables, 850 hPa temperature measured in Kelvin, at the central location in the grid over 4 different forecast horizons: 4, 12, 28, and 120 six-hour steps, corresponding to 1, 3, 7, and 30 days, with context lengths of 28, 28, 56, and 240 observations, respectively. The benchmark uses three separate test periods: 2016, 2017, and 2018. For each experiment, the year immediately preceding the test year is used for validation, while all observations from 1 January 1980 up to the start of the validation year form an expanding training set. All models are trained on the Mean Square Error loss metric.

4.2.1 Forecasting Results

Following the protocol of Shu *et al* [33], performance is assessed for each horizon at the final forecast step using mean absolute error (MAE), Pearson correlation r, and symmetric mean absolute percentage error (sMAPE). For M forecasts with targets y_m and predictions $\widehat{y}_m$,

$$\mathrm{MAE} = \frac{1}{M}\sum_{m=1}^{M} |y_m - \widehat{y}_m|\,, \qquad r = \mathrm{Corr}\,(\mathbf{y}, \widehat{\mathbf{y}})\,.$$

MAE is measured in Kelvin and directly quantifies the average forecasting error. Pearson correlation measures whether the forecasts reproduce variation in the observed temperature series, independently of their absolute scale.

The conventional sMAPE is difficult to interpret for temperatures expressed in Kelvin because the large offset from absolute zero produces artificially small percentage errors. Shu *et al* therefore shift the observed and predicted temperatures before calculating the metric:

$$\mathrm{sMAPE} = \frac{100}{M}\sum_{m=1}^{M} \frac{2\,|y_m - \widehat{y}_m|}{|y_m - \xi| + |\widehat{y}_m - \xi| + 2a},$$

where ξ is the minimum target value in the test set and $a = 30$. Lower MAE and sMAPE indicate better performance, whereas a higher Pearson correlation is preferred.

The Shu *et al* study contains Sonnet and eight competitive neural baseline models: *DLin-*

ear provides a simple linear decomposition baseline; *PatchTST* models each variable independently using temporal patches; *iTransformer* and *Samformer* apply attention across variables; *TimeXer* and *DeformTime* are designed specifically for forecasting with exogenous variables; and *Crossformer* and *ModernTCN* model temporal and inter-variable relationships while preserving the temporal structure of each input series. ModernTCN is particularly important to the discussion because it again provides a direct comparison with GraphTCN and enables us to test whether using spatial mixing for cross series learning provides an advantage over ModernTCN's native variable mixing.

For each combination of city, test year, and forecast horizon, the Sonnet study trained a separate model and selected its hyperparameters using the corresponding validation period. This produces $5 \times 3 \times 4 = 60$ weather forecasting tasks. Note that for our comparison, we do not retrain these baselines and use the results from the Sonnet study. GraphTCN was initialised from the architecture selected on the financial dataset. Since compute was limited, weather-specific hyperparameter selection was restricted to the London dataset using the 2016 train-validation-test split, after which the selected settings were fixed for the remaining cities and test years. The baseline models from the Sonnet study therefore received far more hyperparameter tuning than GraphTCN. The full set of results for GraphTCN are reported in Appendix C. Here, Table 4.10 instead reports the rank achieved by GraphTCN after adding it to the nine models in the Sonnet comparison. Each city-level rank is calculated using the metric averaged across the 2016, 2017, and 2018 test years, matching the presentation used in the main Sonnet results. Rank of one denotes the best-performing model out of the ten.

GraphTCN ranks approximately third overall across all three metrics, despite receiving substantially less weather-specific tuning than the published models. Its performance is also maintained as the forecast horizon increases. In MAE, its strongest mean rank is obtained at the 120-step horizon, indicating that the architecture remains competitive when forecasting 30 days ahead rather than degrading relative to the comparison models.

From Table 4.10, we note that GraphTCN is least competitive in Singapore. A plausible explanation is that the surrounding grid points provide less information about the central target at this location. Figure 4.4 shows the correlation between the central-grid T850 series and each of its eight neighbouring T850 series over the training period. The mean neighbouring correlation is approximately 0.60 in Singapore, compared with values between approximately 0.74 and 0.87 for the other cities. This may explain why GraphTCN performs worst on the Singapore dataset. The model is designed to improve the central forecast by learning how information should be exchanged between spatial locations. When neighbouring series are strongly related to the target, the graph has a clear source of additional predictive information. In Singapore, the weaker spatial infor-

Metric	Horizon	London	New York	Hong Kong	Singapore	Cape Town	Mean rank
MAE	4	2	3	3	5	3	3.2
	12	2	4	4	4	3	3.4
	28	4	2	3	5	3	3.4
	120	3	3	2	3	3	2.8
Pearson	4	2	3	3	5	3	3.2
	12	2	2	3	4	2	2.6
	28	3	2	3	5	3	3.2
	120	3	3	2	5	2	3.0
sMAPE	4	2	3	3	5	3	3.2
	12	3	4	3	4	3	3.4
	28	4	3	2	5	3	3.4
	120	3	3	3	3	3	3.0

Table 4.10: Rank of GraphTCN after inclusion alongside the nine models evaluated by Shu *et al.* Results for each city are averaged across the 2016–2018 test years. Rank one denotes the best-performing model out of ten, and the final column reports the mean rank across cities.

mation reduces this advantage and leaves a greater proportion of the forecasting problem to the temporal module alone.

Table 4.11 provides a more detailed comparison between GraphTCN and four particularly informative baseline models across all 60 tasks. ModernTCN provides the most direct architectural comparison because it uses the same temporal backbone, but learns cross-series relationships through its standard variable-mixing operation. Sonnet, DeformTime and Crossformer are the strongest-performing methods in the published benchmark and therefore show how GraphTCN compares with the leading approaches on this dataset. The first three columns report the number of tasks in which GraphTCN achieves lower MAE, higher Pearson correlation, or lower sMAPE than the comparator. The final column gives relative MAE between GraphTCN and the comparator, computed as $\frac{(\mathrm{MAE}_{\mathrm{GraphTCN}} - \mathrm{MAE}_{\mathrm{comparator}})}{\mathrm{MAE}_{\mathrm{comparator}}}$. This is computed separately for all 60 forecasting tasks and then averaged. Negative values therefore favour GraphTCN.

	Tasks won by GraphTCN			
Comparator	Lower MAE	Higher Pearson r	Lower sMAPE	Relative MAE
ModernTCN	56/60	55/60	56/60	−5.93%
DeformTime	29/60	27/60	25/60	+0.20%
Crossformer	30/60	28/60	28/60	+0.23%
Sonnet	7/60	11/60	7/60	+3.74%

Table 4.11: Task-level comparison of GraphTCN with selected models across the 60 weather forecasting tasks. Comparator results are taken from the Sonnet study.

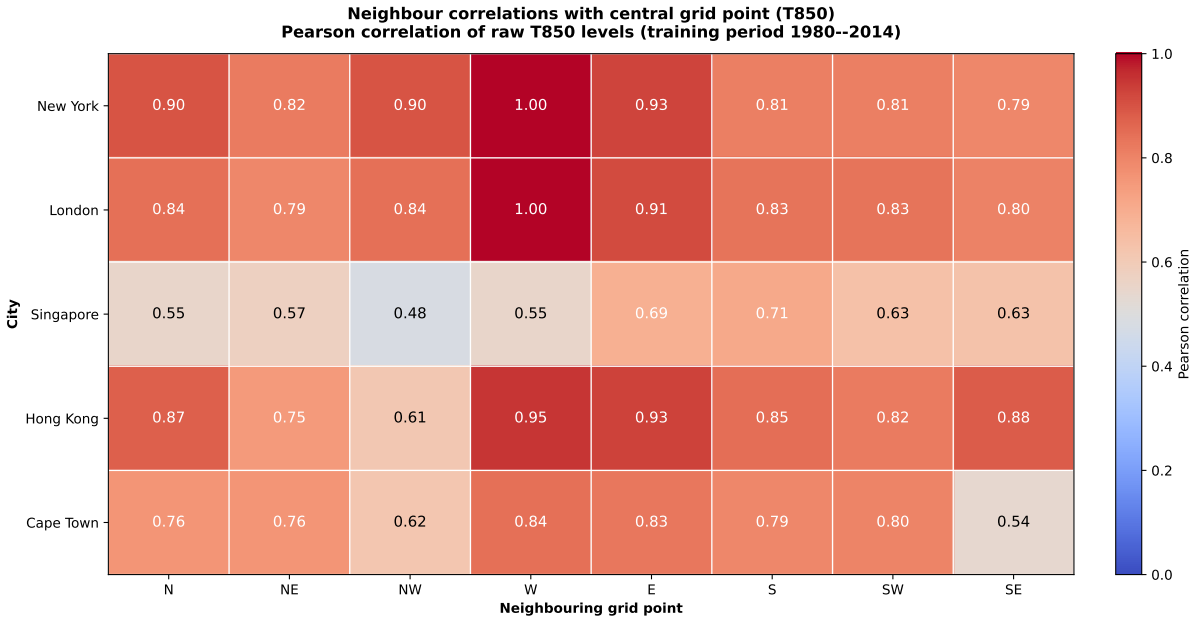


Figure 4.4: Correlation between the T850 target variable at the central grid point and surrounding grid points for each location.

The clearest result is the improvement over ModernTCN. GraphTCN records a lower MAE and sMAPE in 56 of the 60 tasks and a higher Pearson correlation in 55, together with an average MAE reduction of 5.93%. Its MAE advantage also becomes more consistent as the forecasting horizon increases: GraphTCN outperforms ModernTCN in every 7 and 30-day task, with the mean reduction reaching approximately 8.2% at the 30-day horizon. This reinforces the result from the finance dataset and supports the use of an explicit graph layer for cross-series learning rather than ModernTCN's unrestricted variable mixing. By contrast, the comparisons with DeformTime and Crossformer are almost evenly divided. GraphTCN achieves lower MAE in 29 and 30 tasks, respectively, and its average MAE is only 0.20% and 0.23% higher. Its results are therefore comparable with these two strong baselines, despite receiving substantially less weather-specific hyperparameter tuning.

Sonnet remains the clear leading model. Although GraphTCN's average MAE is only 3.74% higher, it achieves a lower MAE in just 7 of the 60 tasks, showing that Sonnet's advantage is highly consistent across locations, years and horizons. This gap may reflect the different inductive biases of the two architectures. Sonnet explicitly represents multi-scale time–frequency structure through learnable wavelets, models inter-variable spectral coherence using MVCA, and applies a Koopman operator to evolve the resulting representation [33]. These components are well suited to weather data containing strong seasonal temporal patterns. GraphTCN instead models cross-location relationships through a learned time-domain adjacency matrix. While this produces slightly weaker forecasting accuracy than Sonnet, it exposes the relationships used during message passing, enabling

the seasonal and event-level graph behaviour examined in Section 4.2.2 to be interpreted directly.

Finally Figure 4.5 provides illustrative GraphTCN forecasts for London and Singapore at the shortest and longest forecast horizons. At the one-day horizon, the forecasts follow much of the short-term variation in the observed T850 series. At the 30-day horizon, the predicted series is substantially smoother in both locations, which is consistent with increasing uncertainty at longer lead times. The Singapore forecasts are visibly more oscillatory than the London forecasts at both horizons. Similar high-frequency variation is present in the observed Singapore series, and GraphTCN reproduces much of this structure, particularly at the one-day horizon. We therefore hypothesise that the more oscillatory appearance primarily reflects the temporal characteristics of the underlying target series rather than instability in the model.

Overall, the weather results provide a clearer demonstration of the value of structured cross-series learning than the financial experiment. GraphTCN consistently improves upon ModernTCN, remains close to the strongest task-specific models, and performs best relative to the comparison set in locations where neighbouring measurements are strongly related to the central target.

4.2.2 Graph Behaviour

Table 4.12 reports the learned values of the α and β gates for each city, forecast horizon, and test year. The value of β is comparatively stable across all experiments, ranging from approximately 0.50 to 0.62, with no clear systematic variation by city, horizon, or year. This mirrors the finance experiment, where β also converged close to 0.5. The model therefore consistently retains substantial contributions from both the original temporal representation and the graph-updated representation, rather than suppressing either pathway.

A much clearer pattern is visible in α. Averaged across cities and test years, α increases from 0.182 at the one-day horizon to 0.260, 0.389, and 0.543 at the three-, seven-, and 30-day horizons, respectively. Moreover, this increase occurs within every city-year combination. Since α controls the weight assigned to the dynamic graph, the shorter-horizon models rely more heavily on the persistent static graph, while the contribution of the dynamic graph increases at longer horizons. One possible explanation is that stable local spatial relationships provide a sufficiently reliable basis for short-term forecasting, whereas the importance of individual neighbouring locations over longer horizons depends more strongly on the current atmospheric configuration, such as prevailing winds,

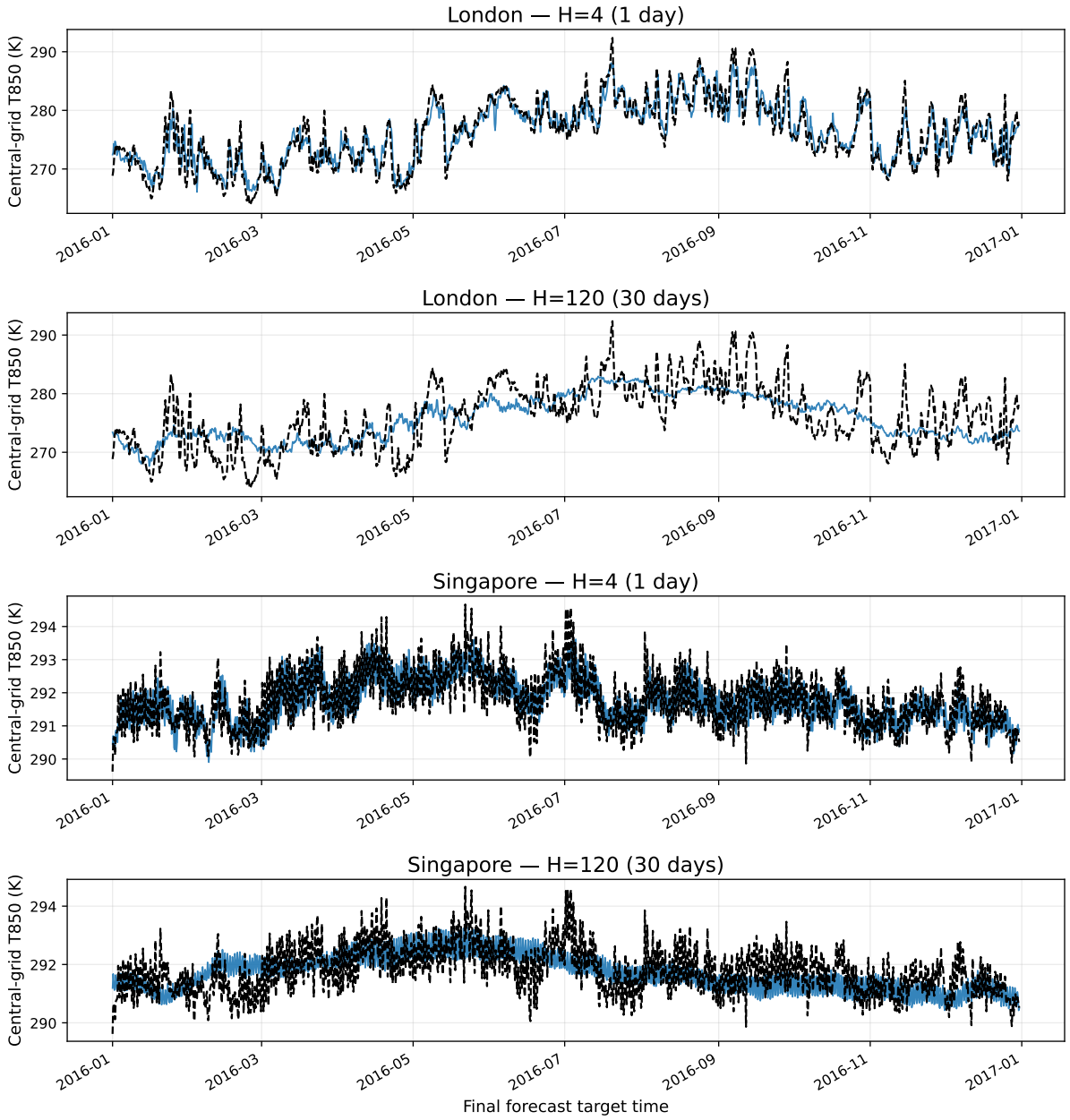


Figure 4.5: Illustrative GraphTCN final-horizon forecasts for London and Singapore during the 2016 test period. From top to bottom, the panels show London at $H = 4$ (one day), London at $H = 120$ (30 days), Singapore at $H = 4$, and Singapore at $H = 120$. Dashed black lines show the observed central-grid T850 series, while solid blue lines show the GraphTCN forecasts.

pressure systems, or spatial temperature gradients. The longer input windows used for the longer-horizon tasks may also provide the graph learner with more evidence from which to infer these dynamic relationships.

		2016		2017		2018	
City	H	α	β	α	β	α	β
Hong Kong	4	0.1731	0.5612	0.1630	0.5303	0.1690	0.5643
	12	0.1995	0.5757	0.1884	0.5426	0.2015	0.5412
	28	0.2171	0.5644	0.2531	0.5535	0.2747	0.5494
	120	0.4751	0.5086	0.4671	0.5121	0.4797	0.5137
Cape Town	4	0.0725	0.6192	0.0774	0.6167	0.0791	0.6131
	12	0.1461	0.5488	0.1059	0.5710	0.0946	0.6144
	28	0.4637	0.5100	0.3681	0.5169	0.1928	0.5428
	120	0.4808	0.5213	0.4739	0.5209	0.4949	0.5391
London	4	0.2855	0.5362	0.3275	0.5643	0.3239	0.5638
	12	0.5290	0.5075	0.3958	0.5510	0.4690	0.5329
	28	0.6003	0.5014	0.5962	0.5208	0.6206	0.5111
	120	0.6916	0.5271	0.6404	0.5111	0.6514	0.5121
New York	4	0.2198	0.5249	0.2586	0.5437	0.2658	0.5469
	12	0.3745	0.5450	0.3747	0.5272	0.3557	0.5441
	28	0.4578	0.5203	0.4433	0.5264	0.4948	0.5136
	120	0.6404	0.5077	0.6162	0.5563	0.6501	0.5095
Singapore	4	0.1032	0.5809	0.1043	0.5848	0.1033	0.5901
	12	0.1419	0.5696	0.1601	0.5709	0.1565	0.5733
	28	0.2929	0.5391	0.2918	0.5947	0.2652	0.5859
	120	0.4514	0.5229	0.4250	0.5992	0.4995	0.5133

Table 4.12: Learned GraphTCN gate values across the weather forecasting experiments. The gate α controls the mixture of the static and dynamic adjacency matrices, while β controls the mixture of the original temporal and graph-updated representations.

Having established that the dynamic component receives greater weight in the longer-horizon models, we next examine individual learned graphs to determine whether their changing edge weights correspond to identifiable weather patterns. To illustrate this context dependence more directly, and following the illustrative examples presented for the finance dataset, Figures 4.6 and 4.7 show how the incoming adjacency weights to the central grid point change under two identifiable weather patterns at the Singapore and Hong Kong location.

Figure 4.6 shows the average graph weights linked to the target central node over the December-February and June-August months for the Singapore location with $H = 28$. We observe that the mean dynamic graph exhibits a clear seasonal reversal. During December-February, the northern grid points $\{\mathrm{NW}, \mathrm{N}, \mathrm{NE}\}$ receive a combined weight

of 0.566, compared with 0.224 for the southern grid points. During June-August, the northern weight falls to 0.202, while the southern weight increases to 0.622. This broadly corresponds to Singapore's seasonal monsoon circulation: the Northeast Monsoon from December to early March is associated with northerly to northeasterly winds, whereas the Southwest Monsoon from June to September is associated with southeasterly to southerly winds [28]. The graph therefore does not retain one fixed spatial weighting, but repeatedly changes the locations from which the central node receives most information according to the prevailing seasonal regime.

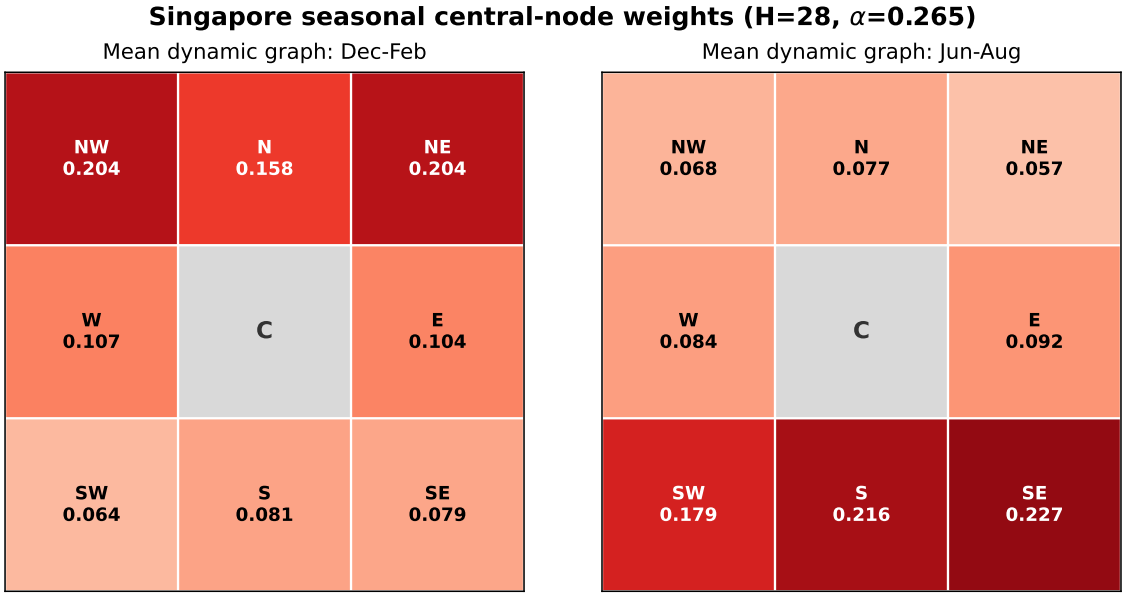


Figure 4.6: Mean dynamic incoming adjacency weights to the central Singapore grid point for December-February and June-August in the $H = 28$ model.

Figure 4.7 provides another example during the passage of Typhoon Vicente near Hong Kong. Typhoon Vicente was a powerful tropical cyclone approached Hong Kong on 23rd Jul 2012. On this date, we observe the dynamic graph assigns almost all of the central node's incoming weight to the southern sector: 0.622 is assigned to the southern grid point, 0.321 to the south-east, and 0.055 to the south-west. After combination with the static graph, these locations still account for 0.697 of the final adjacency weight. This concentration is broadly consistent with Vicente's position to the south of Hong Kong as it approached the coast during 23-24 July 2012 [15].

4.3 Summary

Overall, the results demonstrate that GraphTCN provides a consistent benefit over alternative learned models, although the value of this improvement depends on the pre-

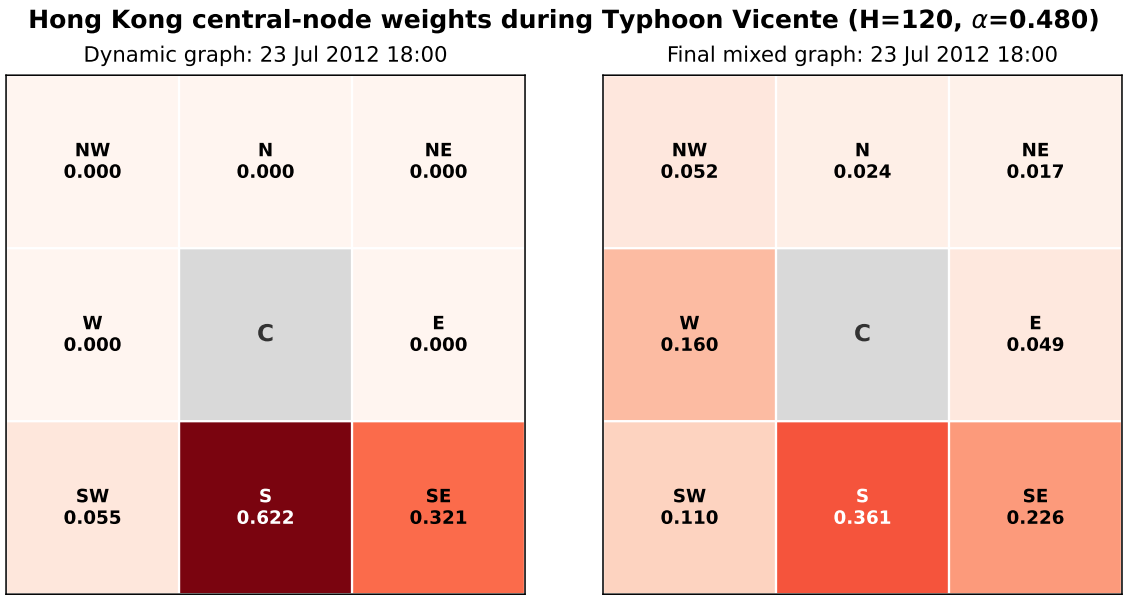


Figure 4.7: Incoming adjacency weights to the central Hong Kong grid point during Typhoon Vicente. The left panel shows the dynamic graph, while the right panel shows the final graph after mixing the dynamic and static components.

dictability and relational structure of the data. On the financial dataset, persistence remained the strongest overall model, but GraphTCN achieved the lowest average Log MAE among the remaining approaches, outperformed ModernTCN at every horizon, and surpassed persistence for the most volatile stocks from 5 to 60 minutes. The token-based models performed poorly in continuous price space despite meaningful token-space predictability, which was attributed to the sensitivity of the Kronos decoder to imperfect token sequences. On the weather benchmark, GraphTCN achieved lower MAE than ModernTCN in 56 of 60 experiments and ranked approximately third among ten models. The learned weather graphs also adapted across horizons, seasonal regimes, and individual weather events, providing further evidence that structured dynamic cross-series learning is most effective when the underlying relationships are stable and informative.

Chapter 5

Conclusion

5.1 Main Findings

This dissertation investigated whether dynamic graph neural networks can improve time-series forecasting by learning when and how information should be exchanged between related series. The principal contribution is GraphTCN, a geometric extension of ModernTCN that separates within-series and cross-series learning. The architecture was evaluated primarily on one-minute data for 93 equities at five forecast horizons and subsequently on a spatial weather benchmark covering five cities, four horizons and three test years.

For the first research question, GraphTCN did not outperform persistence across the complete financial test set. The latest observed price remained the strongest general forecast, reflecting the near-zero mean and weak autocorrelation of short-horizon equity returns. Nevertheless, GraphTCN achieved the lowest average Log MAE among all non-persistence models and was the strongest non-persistence method at the 15-, 30-, and 60-minute horizons. Its clearest advantage occurred for the most volatile quartile of stocks, where it outperformed persistence from 5 to 60 minutes. GraphTCN also produced the highest Pearson correlation between predicted and realised returns at every horizon, although the estimates were small and became uncertain at the longest horizons. The results therefore provide qualified evidence that temporal and cross-stock information can improve forecasts, particularly when volatility creates greater scope to improve upon a no-change prediction.

The second research question considered whether graph-based cross-series learning improves upon ModernTCN's standard unrestricted variable mixing. GraphTCN achieved

lower Log MAE than ModernTCN at every financial horizon and lower MAE in 56 of the 60 weather experiments. This consistent pattern across two substantially different domains supports the use of an explicit graph module to learn cross series relationships over the unrestricted variable mixing of ModernTCN.

For the third research question, the graph analysis showed that GraphTCN uses both persistent and context-dependent structure. In the financial experiment, the static graph retained economically plausible correlation and sector structure, while individual dynamic graphs changed their edge assignments and concentration across forecasting windows. The weather graphs similarly adapted to seasonal circulation and individual weather events. The ablation study found that the complete correlation-initialised static-dynamic formulation achieved the lowest point-estimate Log MAE at every financial horizon, outperforming static-only, dynamic-only, fixed-correlation, and randomly initialised alternatives. Although these differences were small, the combined evidence supports the conclusion that the dynamic component adapts to the observed context and provides additional forecasting value beyond a single static graph.

Finally, the weather benchmark addressed whether GraphTCN generalises to a setting with clearer temporal and spatial structure. GraphTCN ranked approximately third on average among ten evaluated models and consistently improved upon ModernTCN. These results indicate that the architecture can exploit cross-series relationships when they are sufficiently stable and informative. Taken together, the experiments suggest that GraphTCN is a useful and interpretable framework for structured multivariable forecasting, while also showing that architectural improvements cannot fully overcome the limited predictive signal present in short-horizon financial prices.

5.2 Limitations

The principal limitation of the financial experiment is the short calendar span of the dataset. Although one-minute sampling produces a large number of observations, the data covers only a single year, meaning we have a relatively narrow range of market conditions and therefore do not establish whether the learned relationships would remain useful across major regime changes, different liquidity environments, other markets, smaller-cap stocks, or alternative sampling frequencies. Exact external reproduction of the financial experiment is also constrained by the proprietary nature of the dataset.

A second limitation was the available computational budget. Hyperparameter selection for the financial experiment was conducted in stages, with groups of parameters fixed

before subsequent groups were varied, rather than through a complete joint grid search. This may have missed configurations arising from interactions between architectural and optimisation choices. For the weather benchmark, tuning was restricted to one train-validation-test configuration for a single city, after which the selected architectures were transferred to the remaining forecasting tasks. Greater compute would have allowed tuning across a wider range of cities, years, and horizons. It would also have allowed the weather baseline models to be reproduced under the same data-processing and evaluation pipeline, rather than relying on the results reported by Sonnet. Finally, all trained models used a single random seed, so variation due to parameter initialisation and stochastic optimisation could not be quantified.

5.3 Future Work

A natural extension of GraphTCN is probabilistic forecasting. Both experiments in this dissertation produced point estimates, whereas a probabilistic forecasting head could learn a distribution or set of quantiles for every node and forecast horizon, allowing predictive uncertainty and calibration to be evaluated alongside accuracy. The graph module could also be extended in two directions. First, GraphTCN constructs and applies a dense $N \times N$ adjacency matrix, giving quadratic computational and memory complexity as the number of nodes increases. Training on sampled node subgraphs, as proposed in *Connecting the Dots* [38], could enable the model to scale to substantially larger asset universes or spatial networks, although sampling may omit informative relationships between nodes. This could be evaluated by varying the sampled subgraph size and comparing forecasting accuracy, runtime, and memory consumption with the complete graph. Second, the deterministic adjacency could be replaced by a probabilistic latent graph following Neural Relational Inference and its dynamic extension [11, 18]. This would allow GraphTCN to represent uncertainty over the existence or type of each relationship rather than learning a single adjacency matrix, although the resulting stochastic optimisation would be more difficult. Deterministic, static probabilistic, and dynamic probabilistic graph variants could be compared while keeping the temporal backbone fixed.

Tokenisation also remains a promising direction despite the weak continuous-space results obtained in this study. Token GraphTCN demonstrated meaningful predictive power in token space, but the frozen Kronos decoder was highly sensitive to errors in the generated token sequence. Future work could therefore either train a finance-specific tokenizer or jointly post-train the Kronos tokenizer and decoder alongside the forecasting model. In particular, exposing the decoder to model-generated or deliberately corrupted token

sequences during training could make it more robust to the errors encountered at inference time. Although this would require considerably more data and computation and could destabilise the pretrained token representation, it may allow the predictive information captured in token space to translate into more accurate continuous forecasts. A suitable evaluation would compare the frozen tokenizer, decoder-only post-training, and full joint training using reconstruction quality, token accuracy, and final continuous-space forecasting performance.

Bibliography

[1] Hirotugu Akaike. A new look at the statistical model identification. *IEEE transactions on automatic control*, 19(6):716–723, 1974.

[2] Andrew Ang and Joseph Chen. Asymmetric correlations of equity portfolios. *Journal of financial Economics*, 63(3):443–494, 2002.

[3] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2): 157–166, 1994.

[4] Tim Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of econometrics*, 31(3):307–327, 1986.

[5] Michael M Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.

[6] Kyunghyun Cho, Bart Van Merriënboer, Çağlar Gulçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1724–1734, 2014.

[7] Rama Cont. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative finance*, 1(2):223, 2001.

[8] Martin Coulter. Apple in talks to let Google's Gemini power iPhone AI features, Bloomberg News says. Reuters, March 2024. URL `https://www.reuters.com/technology/apple-talks-let-googles-gemini-power-iphone-ai-features-bloomberg-news-says-2024-03-18/`.

[9] Robert F Engle. Autoregressive conditional heteroscedasticity with estimates of the variance of united kingdom inflation. *Econometrica: Journal of the econometric society*, pages 987–1007, 1982.

[10] Eugene F Fama. Efficient capital markets: A review of theory and empirical work. *The journal of Finance*, 25(2):383–417, 1970.

[11] Colin Graber and Alexander G Schwing. Dynamic neural relational inference. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8510–8519. IEEE, 2020.

[12] Peter R Hansen and Asger Lunde. A forecast comparison of volatility models: does anything beat a garch (1, 1)? *Journal of applied econometrics*, 20(7):873–889, 2005.

[13] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

[14] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[15] Hong Kong Observatory. Severe typhoon Vicente. Hong Kong Observatory, July 2012. URL `https://www.hko.gov.hk/en/Press-Releases/100833/Severe-Typhoon-Vicente`.

[16] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 328–339, 2018.

[17] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[18] Thomas Kipf, Ethan Fetaya, Kuan-Chieh Wang, Max Welling, and Richard Zemel. Neural relational inference for interacting systems. In *International conference on machine learning*, pages 2688–2697. Pmlr, 2018.

[19] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[20] Fuxian Li, Jie Feng, Huan Yan, Guangyin Jin, Fan Yang, Funing Sun, Depeng Jin, and Yong Li. Dynamic graph convolutional recurrent network for traffic prediction: Benchmark and solution. *ACM Transactions on Knowledge Discovery from Data*, 17(1):1–21, 2023.

[21] Jingyuan Li, Leo Scholl, Trung Le, Pavithra Rajeswaran, Amy Orsborn, and Eli Shlizerman. Amag: Additive, multiplicative and adaptive graph neural network for forecasting neuron activity. *Advances in Neural Information Processing Systems*, 36: 8988–9014, 2023.

[22] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. *arXiv preprint arXiv:1707.01926*, 2017.

[23] Andrew W Lo and A Craig MacKinlay. Stock market prices do not follow random walks: Evidence from a simple specification test. *The review of financial studies*, 1 (1):41–66, 1988.

[24] Francois Longin and Bruno Solnik. Extreme correlation of international equity markets. *The journal of finance*, 56(2):649–676, 2001.

[25] Donghao Luo and Xue Wang. Moderntcn: A modern pure convolution structure for general time series analysis. In *International Conference on Learning Representations*, 2024.

[26] Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. Statistical and machine learning forecasting methods: Concerns and ways forward. *PloS one*, 13(3):e0194889, 2018.

[27] Dimitri Malandreniotis. Basedygraph: An interlaced dynamic graph benchmark and model for controlled spatio-temporal ablations. Internal report, Autonomous-Fox Laboratories, 2026. Unpublished internal document.

[28] Meteorological Service Singapore. Climate of Singapore. Meteorological Service Singapore Weather Information Portal, N.A. URL `https://www.weather.gov.sg/climate-climate-of-singapore/`.

[29] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. *arXiv preprint arXiv:2211.14730*, 2022.

[30] Mande Praveen, Satish Dekka, Dasari Manendra Sai, Das Prakash Chennamsetty, and Durga Prasad Chinta. Financial time series forecasting: A comprehensive review of signal processing and optimization-driven intelligent models. *Computational Economics*, 67(2):963–989, 2026.

[31] Nabeel Ahmad Saidd. A controlled comparison of deep learning architectures for multi-horizon financial forecasting: Evidence from 918 experiments. *arXiv preprint arXiv:2603.16886*, 2026.

[32] Yu Shi, Zongliang Fu, Shuo Chen, Bohan Zhao, Wei Xu, Changshui Zhang, and Jian Li. Kronos: A foundation model for the language of financial markets. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 25366–25373, 2026.

[33] Yuxuan Shu and Vasileios Lampos. Sonnet: Spectral operator neural network for multivariable time series forecasting. *arXiv preprint arXiv:2505.15312*, 2025.

[34] Robert H Shumway and David S Stoffer. Arima models. In *Time series analysis and its applications: with R examples*, pages 85–175. Springer, 2024.

[35] Jonathan Stempel. Mastercard, Visa reach $30 billion settlement over credit card fees. Reuters, March 2024. URL `https://www.reuters.com/business/finance/mastercard-visa-reach-30-bln-settlement-over-credit-card-fees-2024-03-26/`.

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[37] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

[38] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Xiaojun Chang, and Chengqi Zhang. Connecting the dots: Multivariate time series forecasting with graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 753–763, 2020.

[39] Chao Zhang, Yihuang Zhang, Mihai Cucuringu, and Zhongmin Qian. Volatility forecasting with machine learning and intraday commonality. *Journal of Financial Econometrics*, 22(2):492–530, 2024.

[40] Yue Zhao, Yuanjun Xiong, and Philipp Krähenbühl. Image and video tokenization with binary spherical quantization. In *International Conference on Learning Representations*, volume 2025, pages 90844–90868, 2025.

[41] Han Zhou and Yuntian Chen. Cvtn: Cross variable and temporal integration for time series forecasting. *arXiv preprint arXiv:2404.18730*, 2024.

Appendix A

Code

The source code for GraphTCN along with all training procedures, evaluation metrics and finance baseline models can be found in the GitHub repository:

`https://github.com/VishR-94/GraphTCN.git`

Appendix B

AI Usage Statement

Throughout this thesis, ChatGPT and Codex were used as supplementary tools. Their use was limited to refining wording and phrasing for clarity, assisting with code debugging during experiments, enhancing the visual quality of specific figures, formatting tables, and providing guidance on LaTeX syntax. ChatGPT was also used in the initial literature review stage of the thesis to help find relevant papers, and was used as a learning tool to help understand concepts.

Appendix C

Weather Dataset Full Results

Here we present the full set of evaluation metrics for GraphTCN on the weather benchmark. These can be directly compared with results for all baseline models, which can be found in Shu *et al* [33].

	2016			2017			2018			Average		
H	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**
4	0.9579	0.7154	4.94	0.9628	0.6609	3.67	0.9632	0.6423	4.22	0.9613	0.6729	4.28
12	0.7963	1.4729	9.88	0.8688	1.1900	6.59	0.8394	1.2227	7.78	0.8348	1.2952	8.08
28	0.7307	1.6198	10.77	0.8256	1.3723	7.61	0.7738	1.4374	9.11	0.7767	1.4765	9.16
120	0.7838	1.5434	10.39	0.8070	1.4722	8.15	0.7390	1.6222	10.25	0.7766	1.5459	9.60

Table C.1: GraphTCN forecasting results for Hong Kong.

	2016			2017			2018			Average		
H	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**
4	0.9112	1.6129	9.42	0.8976	1.8688	10.85	0.9088	1.6469	9.93	0.9059	1.7095	10.07
12	0.5437	3.5600	20.34	0.5391	3.7734	21.13	0.5335	3.3800	20.12	0.5388	3.5711	20.53
28	0.4639	3.8004	21.67	0.4797	4.0007	22.35	0.4601	3.5369	20.80	0.4679	3.7793	21.61
120	0.4644	3.8232	21.75	0.5143	3.8959	21.79	0.4189	3.6242	21.29	0.4659	3.7811	21.61

Table C.2: GraphTCN forecasting results for Cape Town.

	2016			2017			2018			Average		
H	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**
4	0.9238	1.6478	10.84	0.8873	1.7670	11.75	0.9226	1.7701	23.17	0.9112	1.7283	15.25
12	0.7562	2.9250	18.44	0.7025	2.8858	18.51	0.7521	3.1930	35.67	0.7369	3.0013	24.20
28	0.6916	3.2813	20.19	0.6161	3.2623	20.74	0.7109	3.5945	39.19	0.6729	3.3794	26.71
120	0.7055	3.2127	19.91	0.6101	3.2833	21.02	0.6935	3.5779	38.74	0.6697	3.3580	26.56

Table C.3: GraphTCN forecasting results for London.

	2016			2017			2018			Average		
H	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**
4	0.9631	1.3398	11.06	0.9624	1.3956	14.98	0.9658	1.2734	10.88	0.9638	1.3363	12.31
12	0.8809	2.3853	19.48	0.8598	2.7325	26.47	0.8903	2.2860	18.80	0.8770	2.4679	21.58
28	0.8565	2.6001	20.74	0.8151	3.0896	29.57	0.8691	2.5021	20.22	0.8469	2.7306	23.51
120	0.8497	2.6416	21.11	0.7779	3.3555	31.07	0.8510	2.6875	20.94	0.8262	2.8949	24.37

Table C.4: GraphTCN forecasting results for New York.

	2016			2017			2018			Average		
H	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**	**r**	**MAE**	**sMAPE**
4	0.8153	0.3723	1.34	0.8104	0.3658	1.33	0.8289	0.3554	1.31	0.8182	0.3645	1.33
12	0.7256	0.4447	1.60	0.7227	0.4410	1.61	0.7232	0.4360	1.60	0.7238	0.4406	1.60
28	0.6302	0.5093	1.83	0.6413	0.4955	1.81	0.6226	0.5013	1.84	0.6314	0.5020	1.83
120	0.6016	0.5321	1.92	0.5666	0.5305	1.93	0.6372	0.4833	1.77	0.6018	0.5153	1.87

Table C.5: GraphTCN forecasting results for Singapore.